



MANTRA DEX

Findings & Analysis Report

2025-03-31

Table of contents

- [Overview](#)
 - [About C4](#)
- [Summary](#)
- [Scope](#)
- [Severity Criteria](#)
- [High Risk Findings \(12\)](#)
 - [\[H-01\] Protocol allows creating broken tri-crypto CPMM pools](#)
 - [\[H-02\] Logical error in `validate\_fees\_are\_paid` can cause a DoS or allow users to bypass fees if `denom\_creation\_fee` includes multiple coins, including `pool\_creation\_fee`, and the user attempts to pay all fees using only `pool\_creation\_fee`](#)
 - [\[H-03\] Multi-token stableswap pools allow `0` liquidity for tokens, creating bricked pools](#)
 - [\[H-04\] Block gas limit can be hit due to loop depth](#)
 - [\[H-05\] Farms can be created to start in past epochs](#)
 - [\[H-06\] Stable swap pools don't properly handle assets with different decimals, forcing LPs to receive wrong shares](#)
 - [\[H-07\] User cannot claim rewards or `close\_position`, due to vulnerable division by zero handling](#)
 - [\[H-08\] Stableswap pool can be skewed free of fees](#)
 - [\[H-09\] Attackers can force the rewards to be stuck in the contract with malicious `x/tokenfactory.denoms`](#)
 - [\[H-10\] Incorrect `slippage\_tolerance` handling in stableswap `provide\_liquidity` function](#)
 - [\[H-11\] Stableswap does disjoint swaps, breaking the underlying invariant](#)
 - [\[H-12\] Pool creators can manipulate the slippage calculation for liquidity providers](#)
- [Medium Risk Findings \(19\)](#)
 - [\[M-01\] In edge cases, `create\_pool` can either be reverted or allow user underpay fees](#)
 - [\[M-02\] Penalty fees can be shared among future farms or expired farms, risks of exploits](#)
 - [\[M-03\] User is unable to claim their reward for the expanded epochs if farm is expanded](#)
 - [\[M-04\] `withdraw\_liquidity` lacks slippage protection](#)
 - [\[M-05\] Insufficient check on asset decimals input in `create\_pool` allows malicious pool to be created with invalid swap results](#)
 - [\[M-06\] Spread calculation does not account for swap fees](#)
 - [\[M-07\] `query\_reverse\_simulation` doesn't account for extra fees when simulating stable reversed swaps](#)
 - [\[M-08\] `compute\_offer\_amount` floors the `offer\_amount` when simulating constant product reversed swaps, leading to unexpected results](#)
 - [\[M-09\] Single sided liquidity can't be used to lock LP tokens in the farm manager](#)
 - [\[M-10\] Protocol fees are mistakenly configured by protocol pools rather than being imposed](#)

- [\[M-11\] When a user single-side deposit into a pool, slippage protection is invalid](#)
- [\[M-12\] Insufficient intermediate value precision in StableSwap calculations](#)
- [\[M-13\] Wrong simulation function used in reverse operation path](#)
- [\[M-14\] Amplifiers can't be ramped allowing loss of funds from the pool](#)
- [\[M-15\] Emergency unlocking penalty makes long duration positions economically advantageous](#)
- [\[M-16\] Liquidity providers can lose tokens due to disproportionate deposits not being properly handled](#)
- [\[M-17\] Slippage tolerance vulnerability in StableSwap](#)
- [\[M-18\] Stablepools return wrong price when they do not converge](#)
- [\[M-19\] Vulnerable liquidity slippage calculation doesn't ensure slippage protection due to unscaled assets sum](#)
- [Low Risk and Non-Critical Issues](#)
 - [Table of Contents](#)
 - [01 Unvalidated genesis epoch update in Epoch Manager allows manipulation of farm rewards](#)
 - [02 Reduced Newton-Raphson iterations easily leads to slightly incorrect results due to potential precision loss](#)
 - [03 Inflation attack protection is not really sufficient](#)
 - [04 Refunds of fees being paid should be processed for all assets](#)
 - [05 Validation of the token factory fee should accept cumulative payments](#)
 - [06 Having a hardcoded maximum slippage should be rethought](#)
 - [07 Wrong counter load while creating positions unnecessarily hikes cost for users creating positions with explicit identifiers](#)
 - [08 Stableswap pools are allowed to have an amp value of 0 which would cause a DOS to swaps on pools](#)
 - [09 Consider allowing a change in the unlocking duration if within valid range when expanding a position](#)
 - [10 Consider upgrading to CosmWasm 2.2.0 for enhanced migration capabilities](#)
 - [11 Allow for the reduction of the `max\_concurrent\_farms` when updating the config](#)
 - [12 Consider enforcing first liquidity provider to be pool creator](#)
 - [13 Wrong pool asset length should be correctly flagged during slippage tolerance assertion](#)
 - [14 Approach of position creation should fail faster when receiver has exceeded their position limit](#)
 - [15 Fix incorrect documentation for position creation authorization check](#)
 - [16 Emergency unlock penalty should not be allowed to be up to 100%](#)
 - [17 Inaccurate year calculation has been set in as AMM constants](#)
 - [18 Inconsistent farm management permissions in sister operations](#)
 - [19 Consider conjugating the pool asset lengths validation when creating a pool](#)
 - [20 Fix typos](#)
 - [21 Remove redundant checks when withdrawing a position](#)
 - [22 Wrong code/documentation about fee](#)
 - [23 Use `assert\_admin` or remove it since we have the `cw-ownable` being used](#)
 - [24 Protocol should be deployment ready](#)
- [Disclosures](#)

[Overview](#)

[About C4](#)

Code4rena (C4) is an open organization consisting of security researchers, auditors, developers, and

individuals with domain expertise in smart contracts.

A C4 audit is an event in which community participants, referred to as Wardens, review, audit, or analyze smart contract logic in exchange for a bounty provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the MANTRA DEX smart contract system. The audit took place from November 29, 2024 to January 13, 2025.

This audit was judged by [3docSec](#).

Final report assembled by Code4rena.

Summary

The C4 analysis yielded an aggregated total of 31 unique vulnerabilities. Of these vulnerabilities, 12 received a risk rating in the category of HIGH severity and 19 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 11 reports detailing issues with a risk rating of LOW severity or non-critical.

All of the issues presented here are linked back to their original finding.

Scope

The code under review can be found within the [C4 MANTRA DEX repository](#), and is composed of 8 smart contracts written in the Rust programming language and includes 22,268 lines of Rust code.

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/non-critical.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

High Risk Findings (12)

[H-01] Protocol allows creating broken tri-crypto CPMM pools

Submitted by [carrotsmuggler](#), also found by [OxAlix2](#), [Abdessamed](#), [DadeKuma](#), [DadeKuma](#), [DadeKuma](#), [gegul](#), [LonnyFlash](#), and [Tigerfrake](#)

[/contracts/pool-manager/src/manager/commands.rs#L75](#)

Finding description and impact

The protocol allows the creation of constant product pools and stableswap pools. Stable-swap pools, as established by curve, can have any number of tokens and so the protocol allows for the creation of pools with 2 or more tokens.

Constant product market-makers (CPMM), however, can have multiple tokens as well; however, the protocol here uses the uniswap formula, which only works for 2-token pools. For pools with more than 2 tokens, this model does not work anymore, and invariants need to be established with different formulas with products of all tokens quantities, like shown in the balancer protocol.

The issue is that the protocol here does not check if the constant product pool being created has more than 2 tokens. Surprisingly, it is perfectly possible to create a constant product pool with 3 tokens, add/remove

liquidity and even do swaps in them, even though the protocol was never designed to handle this.

The POC below will show how we can set up a 3-token CPMM pool, add liquidity and even do swaps in it. The issue is that these pools are completely broken and should not be allowed.

The `compute_swap` function in the `helpers.rs` contract calculates the number of output tokens given the number of input tokens.

```
// ask_amount = (ask_pool * offer_amount / (offer_pool + offer_amount)) - swap_fee - protocol_fee - burn_fee
let return_amount: Uint256 =
    Decimal256::from_ratio(ask_pool.mul(offer_amount), offer_pool + offer_amount)
        .to_uint_floor();
```

But these are only valid for 2-token uniswap-style pools. If there are more than 2 tokens involved, the invariant changes from being $x * y = k$ to $x * y * z = k$, and the formula above does not work anymore. So for multi token pools, this formula should not be used, or $x \rightarrow y$ swaps can be arbitrated off of with $y \rightarrow z$ swaps and vice versa.

Furthermore, there is a check in the `assert_slippage_tolerance` function in the helpers contract:

```
if deposits.len() != 2 || pools.len() != 2 {
    return Err(ContractError::InvalidPoolAssetsLength {
        expected: 2,
        actual: deposits.len(),
    });
}
```

This explicitly shows that constant product pools are only allowed to have 2 tokens. However, if no slippage tolerance is specified, this check can be completely bypassed.

```
pub fn assert_slippage_tolerance(
    slippage_tolerance: &Option<Decimal>,
    deposits: &[Coin],
    pools: &[Coin],
    pool_type: PoolType,
    amount: Uint128,
    pool_token_supply: Uint128,
) -> Result<(), ContractError> {
    if let Some(slippage_tolerance) = *slippage_tolerance {
        // @audit check for number of tokens
    }
}
```

By never sending a slippage tolerance, users can create, add/remove liquidity and even do swaps in pools with more than 2 tokens following constant product algorithm. But these pools are completely broken and should not be allowed since the invariants are not functioning correctly

🔗 Proof of Concept

The POC below creates a CPMM pool with 3 tokens - `uwahle`, `uluna` and `uusd`. It is shown that liquidity can be added and swaps can be performed.

First, some helper functions are needed to check and print out the token balances.

```
fn print_diff(init_bal: [Uint128; 4], final_bal: [Uint128; 4]) -> [i128; 4] {
    let diffs = [
        final_bal[0].u128() as i128 - init_bal[0].u128() as i128,
        final_bal[1].u128() as i128 - init_bal[1].u128() as i128,
        final_bal[2].u128() as i128 - init_bal[2].u128() as i128,
        final_bal[3].u128() as i128 - init_bal[3].u128() as i128,
    ];

    println!("==Balance deltas==");
    if diffs[0] != 0 {
        println!("uwahle delta: {}", diffs[0]);
    }
    if diffs[1] != 0 {
        println!("uluna delta : {}", diffs[1]);
    }
    if diffs[2] != 0 {
        println!("uusd delta : {}", diffs[2]);
    }
}
```

```

    }
    if diffs[3] != 0 {
        println!("lp delta      : {}", diffs[3]);
    }
    println!("==Balance deltas==\n");

    diffs
}

fn calc_state(suite: &mut TestingSuite, creator: &str) -> [Uint128; 4] {
    let uw_hale_balance = RefCell::new(Uint128::zero());
    let uluna_balance = RefCell::new(Uint128::zero());
    let uUSD_balance = RefCell::new(Uint128::zero());
    let lp_shares = RefCell::new(Uint128::zero());

    suite.query_balance(&creator.to_string(), "uw_hale".to_string(), |result| {
        *uw_hale_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_balance(&creator.to_string(), "uluna".to_string(), |result| {
        *uluna_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_balance(&creator.to_string(), "uUSD".to_string(), |result| {
        *uUSD_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_all_balances(&creator.to_string(), |balances| {
        for coin in balances.unwrap().iter() {
            if coin.denom.contains("o.whale.uluna") {
                *lp_shares.borrow_mut() = coin.amount;
            }
        }
    });

    let uw_hale = *uw_hale_balance.borrow();
    let uluna = *uluna_balance.borrow();
    let uUSD = *uUSD_balance.borrow();
    let lp = *lp_shares.borrow();
    [uw_hale, uluna, uUSD, lp]
}

```

And here's the actual test:

► Details

The test runs fine and here's the output:

```

running 1 test
===Liq addition===
==Balance deltas==
uw_hale delta: -1000000
uluna delta : -1000000
uUSD delta  : -1000000
lp delta    : 999000
==Balance deltas==

===Swap===
==Balance deltas==
uw_hale delta: -1000
uUSD delta  : 987
==Balance deltas==

```

It shows:

1. Liquidity addition of `1e6 uw_hale`, `1e6 uluna` `1e6 uUSD` and minting of 999k LP tokens.
2. Swapping `1e3 uw_hale` for `987 uUSD`.

While the swap is correctly functioning here, it doesn't maintain the correct pool invariant and can be arbitrated off of when the pools grow imbalanced.

🔗 Recommended mitigation steps

Add an explicit check during pool creation to make sure constant product pools cannot have more than 2

tokens.

jvrOx (MANTRA) confirmed

🔗 **[H-02] Logical error in `validate_fees_are_paid` can cause a DoS or allow users to bypass fees if `denom_creation_fee` includes multiple coins, including `pool_creation_fee`, and the user attempts to pay all fees using only `pool_creation_fee`**

Submitted by [OxRajkumar](#), also found by [OxAlix2](#), [carrotsmuggler](#), [Egis_Security](#), [Egis_Security](#), [jasonxiale](#), [Lambda](#), [oakcobalt](#), [Tigerfrake](#), [Tigerfrake](#), and [Tigerfrake](#)

[/contracts/pool-manager/src/helpers.rs#L561-L592](#)

🔗 Finding description and Impact

When a user creates a pool, they must pay both `denom_creation_fee` and `pool_creation_fee`.

The `denom_creation_fee` can be paid using multiple coins or a single coin and may also include the same coin as `pool_creation_fee`. If multiple `denom_creation_fee` coins options are available, and one of them matches the coin used for `pool_creation_fee`, it can lead to issues.

🔗 Problem Scenario

The issue arises when the user attempts to pay both fees using the same coin.

1. **Different Fee Amounts:** If the user pays both fees in the same coin, with different amounts for `denom_creation_fee` and `pool_creation_fee`, they might add both amounts and send the total. When validating the `pool_creation_fee`, the check `paid_pool_fee_amount == pool_creation_fee.amount` will fail, causing a DoS.

```
ensure!(
    paid_pool_fee_amount == pool_creation_fee.amount,
    ContractError::InvalidPoolCreationFee {
        amount: paid_pool_fee_amount,
        expected: pool_creation_fee.amount,
    }
);
```

2. **Same Fee Amounts:** If both fees have the same amount and the user pays only once, they can bypass one of the fees entirely, resulting in a fee payment bypass.

```
ensure!(
    paid_pool_fee_amount == pool_creation_fee.amount, //-> HERE It will pass
    ContractError::InvalidPoolCreationFee {
        amount: paid_pool_fee_amount,
        expected: pool_creation_fee.amount,
    }
);

total_fees.push(Coin {
    denom: pool_fee_denom.clone(),
    amount: paid_pool_fee_amount,
});

// Check if the user paid the token factory fee in any other of the allowed denoms
let tf_fee_paid = denom_creation_fee.iter().any(|fee| {
    let paid_fee_amount = info
        .funds
        .iter()
        .filter(|fund| fund.denom == fee.denom)
        .map(|fund| fund.amount)
        .try_fold(Uint128::zero(), |acc, amount| acc.checked_add(amount))
        .unwrap_or(Uint128::zero());
```

```

total_fees.push(Coin {
    denom: fee.denom.clone(),
    amount: paid_fee_amount,
});

paid_fee_amount == fee.amount //-> HERE It will pass
});

```

As both are equal, that's why both checks will pass. The impact is High as it can cause a DoS and allow the bypass of one of the fees.

🔗 Proof of Concept

How it happens when amounts are different:

1. The user sends a transaction to create a pool by combining both fee amounts into a single payment.
2. The transaction reverts because the check `paid_pool_fee_amount == pool_creation_fee.amount` evaluates to false.
3. If the user attempts to bypass this, the next check for `denom_creation_fee` will also fail.

How it happens when amounts are the same:

1. The attacker will send only one amount because both checks (for `pool_creation_fee` and `denom_creation_fee`) will pass, as both amounts are equal. This allows the attacker to pay only once.

🔗 Recommended mitigation steps

We can verify whether the user is paying with one coin or multiple coins. If the user is paying with one coin, we can combine both amounts and perform the validation. Similarly, if the user is paying with multiple coins, we can apply the same approach. This will effectively mitigate the issue.

[jvr0x \(MANTRA\) confirmed and commented:](#)

It is valid. However, considering the chain only supports 1 token to pay for the token factory at the moment, I wouldn't deem it as high, but low.

[3docSec \(judge\) commented:](#)

I see your point, and while I would agree if this were a bug bounty program (funds are not at risk in live contracts), I consider this a High, because what counts is the code in-scope and not the live config, unless the in-scope code is hardcoded to have only one token and can't be changed by config.

🔗 [H-O3] Multi-token stableswap pools allow 0 liquidity for tokens, creating bricked pools

Submitted by [carrotsmuggler](#), also found by [OxAlix2](#), [OxRajkumar](#), [Abdessamed](#), [carrotsmuggler](#), and [LonnyFlash](#)

[/contracts/pool-manager/src/liquidity/commands.rs#L46-L74](#)

[/contracts/pool-manager/src/liquidity/commands.rs#L234-L239](#)

🔗 Finding description and impact

The stableswap pools allow anyone to create pools following the stableswap formula with any number of

tokens. This can be higher than 2. The issue is that the initial `provide_liquidity` does not check if ALL tokens are provided.

The `provide_liquidity` function does a number of checks. For the ConstantProduct pools, the constant product part uses both `deposits[0]` and `deposits[1]` to calculate the initial number of shares, and uses a product of the two. So anyone being absent or 0 leads to reverts during the initial liquidity addition itself.

However, the stableswap pools do not check if the initial liquidity provided is non-zero for all the tokens. So if only 2 of the three tokens are provided, the transaction still goes through. The only check is that all the passed in tokens must be pool constituents.

```
ensure!(
    deposits.iter().all(|asset| pool_assets
        .iter()
        .any(|pool_asset| pool_asset.denom == asset.denom)),
    ContractError::AssetMismatch
);
```

This leads to a broken pool, where further liquidity cannot be added anymore. This is because the pool is saved in a state where the pool has 0 liquidity for one of the tokens. Then in future liquidity additions, `amount_times_coins` value evaluates to 0 for those tokens, which eventually leads to a division by zero error in `d_prod` calculation.

```
let amount_times_coins: Vec<Uint128> = deposits
    .iter()
    .map(|coin| coin.amount.checked_mul(n_coins).unwrap())
    .collect();

// ...

for _ in 0..256 {
    let mut d_prod = d;
    for amount in amount_times_coins.clone().into_iter() {
        d_prod = d_prod
            .checked_mul(d)
            .unwrap()
            .checked_div(amount.into()) //@audit division by zero
            .unwrap();
    }
}

// ...
```

Thus this leads to a broken pool and there is nothing in the contract preventing this.

🔗 Proof of Concept

Attached is a POC where a pool is created with 3 tokens `[uwahle, uluna, uUSD]` but only 2 tokens are provided in the initial liquidity addition `[uwahle, uluna]`. Further liquidity additions revert due to division by zero error.

```
fn multiswap_test() {
    let mut suite = TestingSuite::default_with_balances(
        vec![
            coin(1_000_000_001u128, "uwahle".to_string()),
            coin(1_000_000_000u128, "uluna".to_string()),
            coin(1_000_000_001u128, "uUSD".to_string()),
            coin(1_000_000_001u128, "uom".to_string()),
        ],
        StargateMock::new("uom".to_string(), "8888".to_string()),
    );
    let creator = suite.creator();
    let _other = suite.senders[1].clone();
    let _unauthorized = suite.senders[2].clone();

    let asset_infos = vec![
        "uwahle".to_string(),
        "uluna".to_string(),
        "uUSD".to_string(),
    ];

    // Protocol fee is 0.01% and swap fee is 0.02% and burn fee is 0%
    let pool_fees = PoolFee {
        protocol_fee: Fee {
            share: Decimal::from_ratio(1u128, 1000u128),
        },
    },
```

```

        swap_fee: Fee {
            share: Decimal::from_ratio(1u128, 10_000_u128),
        },
        burn_fee: Fee {
            share: Decimal::zero(),
        },
        extra_fees: vec![],
    };

    // Create a pool
    suite.instantiate_default().create_pool(
        &creator,
        asset_infos,
        vec![6u8, 6u8, 6u8],
        pool_fees,
        PoolType::StableSwap { amp: 100 },
        Some("whale.uluna.uusd".to_string()),
        vec![coin(1000, "uusd"), coin(8888, "uom")],
        |result| {
            result.unwrap();
        },
    );

    // Add liquidity with only 2 tokens
    suite.provide_liquidity(
        &creator,
        "o.whale.uluna.uusd".to_string(),
        None,
        None,
        None,
        None,
        vec![
            Coin {
                denom: "uwhale".to_string(),
                amount: Uint128::from(1_000_000u128),
            },
            Coin {
                denom: "uluna".to_string(),
                amount: Uint128::from(1_000_000u128),
            },
        ],
        |result| {
            result.unwrap();
        },
    );
}

```

The above test passes, showing liquidity can be added with 2 tokens only. Further liquidity provision reverts.

```

// Add liquidity again
suite.provide_liquidity(
    &creator,
    "o.whale.uluna.uusd".to_string(),
    None,
    None,
    None,
    None,
    vec![
        Coin {
            denom: "uwhale".to_string(),
            amount: Uint128::from(1_000_000u128),
        },
        Coin {
            denom: "uluna".to_string(),
            amount: Uint128::from(1_000_000u128),
        },
    ],
    |result| {
        result.unwrap();
    },
);

```

Output:

```

thread 'tests::integration_tests::provide_liquidity::multiswap_test' panicked at contracts/pool-manager/src
called `Result::unwrap()` on an `Err` value: DivideByZeroError

```

```
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
test tests::integration_tests::provide_liquidity::multiswap_test ... FAILED
```

Recommended mitigation steps

Add a check to make sure if total `supply=0`, every token of the pool is provided as liquidity.

jvr0x (MANTRA) confirmed

[H-04] Block gas limit can be hit due to loop depth

Submitted by [carrotsmuggler](#), also found by [OxAlix2](#), [Evo](#), and [Lambda](#)

[/contracts/farm-manager/src/farm/commands.rs#L43-L94](#)

Finding description and impact

The `claim` function iterates over the user positions and calculates the rewards in nested loops. The issue is that every blockchain, to combat against gas attacks of infinite loops, has a block gas limit. If this limit is exceeded, that transaction cannot be included in the chain. The implementation of the `claim` function here is of the order of N^3 and is thus highly susceptible to an out of gas error.

The claim function iterates over all the user's positions.

```
let lp_denoms = get_unique_lp_asset_denoms_from_positions(open_positions);

for lp_denom in &lp_denoms {
    // calculate the rewards for the lp denom
    let rewards_response = calculate_rewards(
        deps.as_ref(),
        &env,
        lp_denom,
        &info.sender,
        current_epoch.id,
        true,
    );
    //...
}
```

Lets say the user has P positions, all of different `lp_denom` values. Thus this loop is of the order of P . The `calculate_rewards` function then loops over all the farms of each `lp_denom`.

```
let farms = get_farms_by_lp_denom(
    deps.storage,
    lp_denom,
    None,
    Some(config.max_concurrent_farms),
);
//...
for farm in farms {
    // skip farms that have not started
    if farm.start_epoch > current_epoch_id {
        continue;
    }

    // compute where the user can start claiming rewards for the farm
    let start_from_epoch = compute_start_from_epoch_for_address(
        deps.storage,
        &farm.lp_denom,
        last_claimed_epoch_for_user,
        receiver,
    );
    //...
}
```

Say there are F farms, then this inner loop is of the order of F . Then for each farm, the reward is calculated by iterating over all the epochs from `start_from_epoch` up to the `current_epoch`.

```
for epoch_id in start_from_epoch..until_epoch {
    if farm.start_epoch > epoch_id {
```

```

        continue;
    }
    //...
}

```

The `start_from_epoch` can be the very first deposit of the user, far back in time, if this is the first time the user is claiming rewards. Thus, this loop can run very long if the position is years old. Say the epoch loop is of the order of E .

Since these 3 loops are nested, the `claim` function is of the order of $P * F * E$. P and F are restricted by the config can have maximum values of the order of 10. But E can be very large, and is actually the order of epoch number. So if epochs are only a few days long, the E can be of the order of 500 over a couple of years.

Thus the `claim` function can be of the order of `50_000`. This is an issue since it requires a loop running `50_000` times along with reward calculations and even token transfers. This can be above the block gas limit and thus the transaction will fail.

There is no functionality to skip positions/farms/epochs. Thus users cannot claim rewards of only a few particular farms or epochs. This part of the code is also executed during the `close_position` function, which checks if rewards are 0. Thus, the `close_position` function can also fail due to the same issue, and users are thus forced to emergency withdraw and lose deposits as well as their rewards.

Thus users who join a bunch of different farms and keep their positions for a long time can hit the block gas limit during the time of claiming rewards or closing positions.

The OOG issue due to large nesting depth is present in multiple instances in the code, this is only one example.

Proof of Concept

P , the number of open positions of a user, is restricted by limit of 100 stored in `MAX_ITEMS_LIMIT`. F is restricted by the max concurrent no of farms per `lp_denom`, which we can assume to be 10. E is of the order of epochs between the first deposit and the current epoch, which can be in the 100s if epochs are single days, or 100s if epochs are weeks.

Thus, $P * F * E$ is of the order of $100 * 10 * 100 = 100_000$; `100_000` iterations are required for the `claim` function on top of token transfers and math calculations. This can easily exceed the block gas limit.

Recommended mitigation steps

The order of the nested loops need to be decreased. This can be done in multiple ways.

1. Implement sushi-masterchef style reward accounting. This way the entire E number of epochs don't need to be looped over.
2. Implement a way to only process a given number of positions. This way P can also be restricted and users can claim in batches.

jvr0x (MANTRA) confirmed

[H-05] Farms can be created to start in past epochs

Submitted by [Abdessamed](#), also found by [OxAlix2](#), [carrotsmuggler](#), [Lambda](#), and [Tigerfrake](#)

[/contracts/farm-manager/src/helpers.rs#L128-L175](#)

Finding description and impact

In the farming mechanism, users can claim rewards from active farms based on their locked LP token share. The rewards distribution must adhere to the following invariant:

At any given epoch, all users with locked LP tokens claim rewards from corresponding farms proportional to their share of the total LP tokens.

However, the current implementation allows the creation of farms with a `start_epoch` in the past. This breaks the invariant, as users who have already claimed rewards for past epochs will miss out on additional rewards assigned retroactively to those epochs. This issue arises because the `validate_farm_epochs` function does not enforce that the farm's start epoch must be in the future relative to the current epoch:

```
/// Validates the farm epochs. Returns a tuple of (start_epoch, end_epoch) for the farm.
pub(crate) fn validate_farm_epochs(
    params: &FarmParams,
    current_epoch: u64,
    max_farm_epoch_buffer: u64,
) -> Result<(u64, u64), ContractError> {
    let start_epoch = params.start_epoch.unwrap_or(current_epoch + 1u64);

    ensure!(
        start_epoch > 0u64,
        ContractError::InvalidEpoch {
            which: "start".to_string()
        }
    );

    let preliminary_end_epoch = params.preliminary_end_epoch.unwrap_or(
        start_epoch
            .checked_add(DEFAULT_FARM_DURATION)
            .ok_or(ContractError::InvalidEpoch {
                which: "end".to_string(),
            })?,
    );

    // ensure that start date is before end date
    ensure!(
        start_epoch < preliminary_end_epoch,
        ContractError::FarmStartTimeAfterEndTime
    );

    // ensure the farm is set to end in a future epoch
    ensure!(
        preliminary_end_epoch > current_epoch,
        ContractError::FarmEndsInPast
    );

    // ensure that start date is set within buffer
    ensure!(
        start_epoch
            <= current_epoch.checked_add(max_farm_epoch_buffer).ok_or(
                ContractError::OverflowError(OverflowError {
                    operation: OverflowOperation::Add
                })
            )?,
        ContractError::FarmStartTooFar
    );

    Ok((start_epoch, preliminary_end_epoch))
}
```

The function lacks a check to ensure that `start_epoch` is not earlier than `current_epoch + 1`, allowing farms to be created retroactively. This leads to unfair rewards distribution.

Proof of Concept

The following test case demonstrates that a farm can be created in such a way that it starts in a past epoch, copy and paste the following test to `/contracts/farm-manager/tests/integration.rs`:

```
#[test]
fn poc_farm_can_be_created_in_the_past() {
    let lp_denom = format!("factory/{MOCK_CONTRACT_ADDR_1}/{LP_SYMBOL}").to_string();
    let invalid_lp_denom = format!("factory/{MOCK_CONTRACT_ADDR_2}/{LP_SYMBOL}").to_string();

    let mut suite = TestingSuite::default_with_balances(vec![
        coin(1_000_000_000u128, "uom".to_string()),
        coin(1_000_000_000u128, "uusdy".to_string()),
        coin(1_000_000_000u128, "uosmo".to_string()),
        coin(1_000_000_000u128, lp_denom.clone()),
        coin(1_000_000_000u128, invalid_lp_denom.clone()),
    ])
```

```

]);
suite.instantiate_default();

let creator = suite.creator().clone();
let other = suite.senders[1].clone();
let fee_collector = suite.fee_collector_addr.clone();

for _ in 0..10 {
    suite.add_one_epoch();
}
// current epoch is 10

// We can create a farm in a past epoch
suite
    .manage_farm(
        &other,
        FarmAction::Fill {
            params: FarmParams {
                lp_denom: lp_denom.clone(),
                start_epoch: Some(1), // @audit Notice, start epoch in the past
                preliminary_end_epoch: Some(28),
                curve: None,
                farm_asset: Coin {
                    denom: "uusdy".to_string(),
                    amount: Uint128::new(4_000u128),
                },
                farm_identifier: Some("farm_1".to_string()),
            },
        },
        vec![coin(4_000, "uusdy"), coin(1_000, "uom")],
        |result| {
            result.unwrap();
        },
    );
}

```

The transaction passes without reverting, creating a farm that starts in a past epoch.

🔗 Recommended mitigation steps

Ensure the `start_epoch` is always in the future relative to the `current_epoch`:

```

/// Validates the farm epochs. Returns a tuple of (start_epoch, end_epoch) for the farm.
pub(crate) fn validate_farm_epochs(
    params: &FarmParams,
    current_epoch: u64,
    max_farm_epoch_buffer: u64,
) -> Result<(u64, u64), ContractError> {
    let start_epoch = params.start_epoch.unwrap_or(current_epoch + 1u64);
    + assert!(start_epoch >= current_epoch + 1);
    // --SNIP
}

```

jvr0x (MANTRA) confirmed

🔗 [H-06] Stable swap pools don't properly handle assets with different decimals, forcing LPs to receive wrong shares

Submitted by [OxAlix2](#), also found by [Ox1982us](#), [Abdessamed](#), [carrotsmuggler](#), and [oakcobalt](#)

Stable swap pools in Mantra implement Curve's stable swap logic, this is mentioned in the [docs](#). Curve normalizes the tokens in a stable swap pool, by having something called rate multipliers where they're used to normalize the tokens' decimals. This is critical as it is used in D computation [here](#).

The reflection of this in Mantra is `compute_d`, where it does something similar, [here](#):

```

// sum(x_i), a.k.a S
let sum_x = deposits
    .iter()
    .fold(Uint128::zero(), |acc, x| acc.checked_add(x.amount).unwrap());

```

However, the issue is that amounts are not normalized from the caller, where this is called from

```
compute_lp_mint_amount_for_stableswap_deposit:
```

```
#[allow(clippy::unwrap_used, clippy::too_many_arguments)]
pub fn compute_lp_mint_amount_for_stableswap_deposit(
    amp_factor: &u64,
    old_pool_assets: &[Coin],
    new_pool_assets: &[Coin],
    pool_lp_token_total_supply: Uint128,
) -> Result<Option<Uint128>, ContractError> {
    // Initial invariant
    @> let d_0 = compute_d(amp_factor, old_pool_assets).ok_or(ContractError::StableInvariantError)?;

    // Invariant after change, i.e. after deposit
    // notice that new_pool_assets already added the new deposits to the pool
    @> let d_1 = compute_d(amp_factor, new_pool_assets).ok_or(ContractError::StableInvariantError)?;

    // If the invariant didn't change, return None
    if d_1 <= d_0 {
        Ok(None)
    } else {
        let amount = Uint512::from(pool_lp_token_total_supply)
            .checked_mul(d_1.checked_sub(d_0)?)?
            .checked_div(d_0)?;
        Ok(Some(Uint128::try_from(amount)?))
    }
}
```

This messes up the whole shares calculation logic, as D would be way greater for LPs depositing tokens of higher decimals than other tokens in the same stable swap pool.

NB: This is handled for swaps, [here](#).

🔗 Proof of Concept

Add the following in `contracts/pool-manager/src/tests/integration_tests.rs`:

The following test creates a stable swap pool with 3 assets, 2 of them have 6 decimals, while the 3rd has 18 decimals. Initially, the same amount `*` asset decimals of each asset is deposited, depositing the same amount of the 18 decimal token results in an exaggerated amount of shares minted to the LP.

To double check this, you can try changing `uweth`'s decimals to 6, and confirm that both test cases result in equal number of shares, unlike the current implementation, where the difference is huge.

► Details

🔗 Recommended mitigation steps

Whenever computing D, make sure all the deposits/amounts are in the “non-decimal” value, i.e., without decimals. For example, `100e6` should just be sent as 100, just like how it's done in `compute_swap`. This should be added in `compute_d`.

jvr0x (MANTRA) confirmed

🔗 [H-07] User cannot claim rewards or `close_position`, due to vulnerable division by zero handling

Submitted by [oakcobalt](#), also found by [OxAlix2](#), [Daniel526](#), [Lambda](#), and [Tigerfrake](#)

A user cannot claim rewards or `close_position`, due to vulnerable division by zero handling in the `claim -> calculate_rewards` flow.

In `calculate_rewards`, a user's reward per farm per epoch is based on the `user_share` `(user_weight / contract_weights)`; `contract_weights` can be zero.

The main vulnerability is division by zero handling is not done at the site of division; i.e., no check on `contract_weights` is non-zero before using it as a denominator in `checked_mul_floor`. (Flows: `claim -> calculate_rewards`).

```
//contracts/farm-manager/src/farm/commands.rs
```

```
pub(crate) fn calculate_rewards(
...
) -> Result<RewardsResponse, ContractError> {
...
    for epoch_id in start_from_epoch..=until_epoch {
...
        let user_weight = user_weights[epoch_id];
        let total_lp_weight = contract_weights
            .get(&epoch_id)
            .unwrap_or(&Uint128::zero())
            .to_owned();
        // @audit contract_weights or total_lp_weight can be zero, when used as a fraction with checked_
    |> let user_share = (user_weight, total_lp_weight);

        let reward = farm_emissions
            .get(&epoch_id)
            .unwrap_or(&Uint128::zero())
            .to_owned()
    |> .checked_mul_floor(user_share)?;
...
}
```

[/contracts/farm-manager/src/farm/commands.rs#L205](#)

Current contract attempts to handle this at the source; clear the users `LAST_CLAIMED_EPOCH` when a user closes a position. This is also vulnerable because when the user has active positions in other lp-denoms, `LAST_CLAIMED_EPOCH` cannot be cleared for the user. Back in `calculate_rewards`, this means the epoch iteration will still start at (`LAST_CLAIMED_EPOCH + 1`) which includes the epoch where `contract_weights` is zero. (Flows: `close_position -> reconcile_user_state`).

```
// contracts/farm-manager/src/position/helpers.rs
pub fn reconcile_user_state(
    deps: DepsMut,
    receiver: &Addr,
    position: &Position,
) -> Result<(), ContractError> {
    let receiver_open_positions = get_positions_by_receiver(
        deps.storage,
        receiver.as_ref(),
        Some(true),
        None,
        Some(MAX_ITEMS_LIMIT),
    );

    // if the user has no more open positions, clear the last claimed epoch
    // @audit-info note: LAST_CLAIMED_EPOCH will not be cleared for the user when the user has open position
    if receiver_open_positions.is_empty() {
    |> LAST_CLAIMED_EPOCH.remove(deps.storage, receiver);
    }
    ...
}
```

[/contracts/farm-manager/src/position/helpers.rs#L215](#)

🔗 Impact

Users' rewards will be locked and unclaimable. Since [pending rewards have to be claimed](#) before `close_position`, users cannot close any positions without penalty.

🔗 Proof of Concept

Suppose a user has two positions: position1 (`lp_denom1`) and position2(`lp_denom2`).

1. User calls `close_position` to close position1.
2. In `close_position` → `update_weights`, contract weight for `lp_denom1` becomes 0 in the following epoch.
3. In `close_position` → `reconcile_user_state`, `LAST_CLAIMED_EPOCH.remove` is skipped due to user has other positions.
4. After a few epochs, user create position3 (`lp_denom1`).
5. After a few epochs, user calls claims. `claim tx` reverts due to division by zero.

6. Now, all of the rewards of the user are locked.

Coded PoC:

In `contracts/farm-manager/tests/integration.rs`, add

```
test_query_rewards_divide_by_zero_cause_rewards_locked and run cargo test
test_query_rewards_divide_by_zero_cause_rewards_locked.
```

► Details

```
running 1 test
test test_query_rewards_divide_by_zero_cause_rewards_locked ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 40 filtered out; finished in 0.01s
```

🔗 Recommended mitigation steps

Consider handling division by zero in `calculate_rewards` directly by skip the epoch iteration when `contract_weights` is 0.

jvr0x (MANTRA) confirmed

🔗 [H-08] Stableswap pool can be skewed free of fees

Submitted by [carrotsmuggler](#), also found by [Abdessamed](#)

[/contracts/pool-manager/src/liquidity/commands.rs#L257-L265](#)

🔗 Finding description and impact

Stableswap pools are designed to work around a set pricepoint. If the price of the pool deviates away from that point, the pool can incur large slippage.

Normally in constant product AMMs (CPMMs), slippage is observed at every point in the curve. However, for a user to change the price drastically, they need to do a large swap. This costs them swap fees. In CPMMs, adding liquidity does not change the price since they always have to be added at specific ratios.

For stableswaps, this is not true. In stableswap pools, liquidity can be added in at any ratio. This means that a user can add liquidity at a ratio far from the current price, which will change the ratio of funds in the pool, leading to large slippage for all users. Stableswap pools protect against this by using fees.

If we look at the curve protocol, we see that if liquidity is added at a ratio far from the current price, the `difference` between the liquidity addition price and ideal price is computed. The contract can be found [here](#).

```
ideal_balance = D1 * old_balances[i] / D0
difference = 0
new_balance = new_balances[i]

if ideal_balance > new_balance:
    difference = unsafe_sub(ideal_balance, new_balance)
else:
    difference = unsafe_sub(new_balance, ideal_balance)
```

This basically is a measure of how much the pool is being skewed due to this liquidity addition. The user is then made to pay swap fees for this `skew` they introduced.

```
_dynamic_fee_i = self._dynamic_fee(xs, ys, base_fee)
fees.append(unsafe_div(_dynamic_fee_i * difference, FEE_DENOMINATOR))
self.admin_balances[i] += unsafe_div(fees[i] * admin_fee, FEE_DENOMINATOR)
new_balances[i] -= fees[i]
```

So, If a user adds liquidity at the current pool price, `difference` will be 0 and they wont be charged fees. But if they add liquidity at a skewed price, they will be charged a fee which is equal to the swap fee on the skew they introduced.

This basically makes them equivalent to CPMs, where to change the price you need to pay swap fees. In stableswap pools like on curve, you pay swap fees if you change the price during liquidity addition.

The issue is that in the stableswap implementation in the codebase, this fee isn't charged. So users skewing the stableswap pool can basically do it for free, pay no swap fees and only lose out on some slippage.

```
let d_0 = compute_d(amp_factor, old_pool_assets).ok_or(ContractError::StableInvariantError)?;
let d_1 = compute_d(amp_factor, new_pool_assets).ok_or(ContractError::StableInvariantError)?;

if d_1 <= d_0 {
    Ok(None)
} else {
    let amount = Uint512::from(pool_lp_token_total_supply)
        .checked_mul(d_1.checked_sub(d_0)?)?
        .checked_div(d_0)?;
    Ok(Some(Uint128::try_from(amount)?))
}
```

Here `new_pool_assets` is just `old_pool_assets` + `deposits`. So a user can add liquidity at any ratio, and not the penalty for it. This can be used by any user to manipulate the pool price or highly skew the pool composition.

Attached is a POC showing a user doing the same. This shows that the pools can be manipulated very easily at very low costs, and users are at risk of losing funds due to high slippage.

Proof of Concept

In the following POC, the following steps take place:

1. A 2-token stableswap pool is created with `uwahle` and `uluna`.
2. `1e6` of each token is added as liquidity. This is `Liq addition` event.
3. A user adds `2e6` of `uwahle` and no `uluna`. This is `Liq addition 2`.
4. The user then removes the liquidity. This is `Liq removal`.

First let's look at the output from the POC:

```
running 1 test
===Liq addition===
==Balance deltas==
uwahle delta: -1000000
uluna delta : -1000000
lp delta    : 1999000
==Balance deltas==

===Liq addition 2===
==Balance deltas==
uwahle delta: -2000000
lp delta    : 1993431
==Balance deltas==

===Liq Removal===
==Balance deltas==
uwahle delta: 1497532
uluna delta : 499177
lp delta    : -1993431
==Balance deltas==
```

Lets assume `uwahle` and `uluna` are both 1 USD each. In step 3, the user added `2e6` of `whale`, so added in `2e6` `usd`. In step 4, the user removed `1497532+499177=1996709` `usd`. So the user recovered 99.84% of their funds. They only lost 0.16% to slippage.

In step 4 we see the impact. When the user removes liquidity, the liquidity comes out at a ratio of `1:0.33`. Thus, the pool is highly skewed and the price of the pool will be reported incorrectly and users will incur high slippage.

So at the cost of just 0.16% of their funds, the user was able to skew the pool by a massive amount. Even though the swap fees are 10% in the POC, the user was able to do this paying only 0.16%. This level of low-cost manipulation would be impossible even for CPMM pools, which are known to be more manipulatable than stableswap pools.

Below are some of the helper functions used in the POC:

```
fn print_diff(init_bal: [Uint128; 4], final_bal: [Uint128; 4]) -> [i128; 4] {
    let diffs = [
        final_bal[0].u128() as i128 - init_bal[0].u128() as i128,
        final_bal[1].u128() as i128 - init_bal[1].u128() as i128,
        final_bal[2].u128() as i128 - init_bal[2].u128() as i128,
        final_bal[3].u128() as i128 - init_bal[3].u128() as i128,
    ];

    println!("==Balance deltas==");
    if diffs[0] != 0 {
        println!("uwahle delta: {}", diffs[0]);
    }
    if diffs[1] != 0 {
        println!("uluna delta : {}", diffs[1]);
    }
    if diffs[2] != 0 {
        println!("uusd delta : {}", diffs[2]);
    }
    if diffs[3] != 0 {
        println!("lp delta : {}", diffs[3]);
    }
    println!("==Balance deltas==\n");

    diffs
}

fn calc_state(suite: &mut TestingSuite, creator: &str) -> [Uint128; 4] {
    let uwahle_balance = RefCell::new(Uint128::zero());
    let uluna_balance = RefCell::new(Uint128::zero());
    let uusd_balance = RefCell::new(Uint128::zero());
    let lp_shares = RefCell::new(Uint128::zero());

    suite.query_balance(&creator.to_string(), "uwahle".to_string(), |result| {
        *uwahle_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_balance(&creator.to_string(), "uluna".to_string(), |result| {
        *uluna_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_balance(&creator.to_string(), "uusd".to_string(), |result| {
        *uusd_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_all_balances(&creator.to_string(), |balances| {
        for coin in balances.unwrap().iter() {
            if coin.denom.contains("o.wahle.uluna.uusd") {
                *lp_shares.borrow_mut() = coin.amount;
            }
        }
    });

    let uwahle = *uwahle_balance.borrow();
    let uluna = *uluna_balance.borrow();
    let uusd = *uusd_balance.borrow();
    let lp = *lp_shares.borrow();
    [uwahle, uluna, uusd, lp]
}
```

Attached is the full POC code.

► Details

🔗 Recommended mitigation steps

Similar to curve, add swap fees based on the skewness introduced in the stableswap pools during liquidity addition.

jvr0x (MANTRA) confirmed

🔗 [H-09] Attackers can force the rewards to be stuck in the contract with malicious `x/tokenfactory` denoms

Submitted by [peachtea](#), also found by [Audinarey](#), [carrotsmuggler](#), [Egis Security](#), and [pOwd3r](#)

Attackers can fund rewards of LP tokens with tokens created from the `x/tokenfactory` module and abuse the `MsgForceTransfer` message to prevent the contract from successfully distributing rewards. This would also prevent the contract owner from closing the malicious farm. As a result, rewards that are accrued to the users will be stuck in the contract, causing a loss of rewards.

🔗 Proof of Concept

When a user claims pending rewards of their LP tokens, all of their rewards are aggregated together and sent within a `BankMsg::Send` message.

[/contracts/farm-manager/src/farm/commands.rs#L102-L107](#)

These rewards can be funded externally via the `FarmAction::Fill` message for a particular LP asset.

One thing to note is that the reward must be a `Coin`, which means it must be a native token recognized by the Cosmos SDK module.

https://github.com/code-423n4/2024-11-mantra-dex/blob/26714ea59dab7ecfaca9db1138d60adcf513588/packages/amm/src/farm_manager.rs#L186-L187

The Mantra DEX contract will be deployed in the Mantra chain, which is running in parallel as another competition [here](#). The Mantra chain implements a `x/tokenfactory` module to allow token creators to create native tokens.

https://github.com/MANTRA-Chain/mantrachain/blob/v1.0.2/x/tokenfactory/keeper/msg_server.go

One of the features in the `x/tokenfactory` module is that token creators can call the `MsgForceTransfer` to forcefully transfer funds from one account to another account, effectively reducing its balance.

https://github.com/MANTRA-Chain/mantrachain/blob/v1.0.2/x/tokenfactory/keeper/msg_server.go#L149

This allows an attacker to perform a denial of service of the rewards pending in the contract by supplying a tokenfactory denom, and then forcefully transfer funds from the contract in order to cause an “insufficient funds” error.

1. The attacker creates an `x/tokenfactory` denom from the Mantra chain.
2. The attacker mints some of the tokens and supplies them to an LP token with `FarmAction::Fill`.
3. The attacker calls `MsgForceTransfer` to transfer all the tokens forcefully from the contract.
4. When users want to claim their rewards, the transaction will fail due to an insufficient funds error. Since all the rewards are aggregated into a single `BankMsg::Send`, other legitimate rewards that are accrued for the user will be stuck and cannot be withdrawn.
5. At this point, the contract owner notices it and sends the `FarmAction::Close` messages to close the farm created by the attacker. However, because the `close_farms` function will automatically refund the unclaimed `farm.farm_asset.amount` to the attacker (see [here](#)), the transaction will fail due to an insufficient funds error.

🔗 Recommended mitigation steps

To mitigate this attack, consider modifying the `close_farms` function so the messages are dispatched as `SubMsg::reply_on_error` when refunding the rewards to the farm owner. Within the reply handler, simply return an `Ok(Response::default())` if an error occurred during `BankMsg::Send`. This will prevent the attack because the contract owner will still have the power to close malicious farms even though the attacker reduced the contract's balance.

https://docs.rs/cosmwasm-std/latest/cosmwasm_std/struct.SubMsg.html#method.reply_on_error

jvr0x (MANTRA) confirmed

[3docSec \(judge\) commented:](#)

Marking this one as primary, because it highlights the two impacts in this group:

- Malicious pools brick claiming of legitimate pools' rewards.
- Malicious pools can't be closed.

It is, however, recommended to take into consideration also the [S-377](#) mitigation of letting users opt-out from malicious pools without requiring admin intervention

[\[H-10\] Incorrect `slippage\_tolerance` handling in `stableswap` `provide\_liquidity` function](#)

Submitted by [carrotsmugger](#), also found by [oakcobalt](#)

[/contracts/pool-manager/src/helpers.rs#L437-L438](#)

Finding description and impact

The `provide_liquidity` function is used to add liquidity to the dex pools. This function implements a slippage tolerance check via the `assert_slippage_tolerance` function.

```
helpers::assert_slippage_tolerance(  
    &slippage_tolerance,  
    &deposits,  
    &pool_assets,  
    pool.pool_type.clone(),  
    share,  
    total_share,  
)?;
```

This function implements slippage tolerance in two sub-functions, one for stableswap and one for constant product. This function basically compares the ratio of liquidity of the deposit to the ratio of pool liquidity.

For the constant product pool, the slippage tolerance checks both the 1/O ratio and O/1 ratios, where O and 1 represent the two tokens of the pool.

```
if Decimal256::from_ratio(deposits[0], deposits[1]) * one_minus_slippage_tolerance  
    > Decimal256::from_ratio(pools[0], pools[1])  
|| Decimal256::from_ratio(deposits[1], deposits[0])  
    * one_minus_slippage_tolerance  
    > Decimal256::from_ratio(pools[1], pools[0])  
{  
    return Err(ContractError::MaxSlippageAssertion);  
}
```

But for stableswap, it only does a one-sided check.

```
if pool_ratio * one_minus_slippage_tolerance > deposit_ratio {  
    return Err(ContractError::MaxSlippageAssertion);  
}
```

The situation is best described for the scenario where `slippage_tolerance` is set to 0. This means the pool should ONLY accept liquidity in the ratio of the pool liquidity. This is enforced for constant product pools correctly. However, for stableswap pools, this is incorrect.

If `slippage_tolerance` is set to 0, then `one_minus_slippage_tolerance` is 1. Thus, the inequality check above makes sure that the `pool_ratio` is always less than or equal to the `deposit_ratio` for the transaction to go through. However, the `deposit_ratio` can be either higher or lower than the `pool_ratio`, depending on the components of the liquidity addition. The inequality above only checks for one case (less

than equals) and misses the other check (greater than equals).

This means even with `slippage_tolerance` set to 0, the stableswap pool will accept liquidity that is not in the ratio of the pool liquidity.

Furthermore, for the case where the `deposit_ratio` is higher than the `pool_ratio`, there is no slippage restriction on the pool at all.

The entire reason `slippage_tolerance` exists, is so that the user can specify the exact amount of lp tokens they expect out of the pool. However, the protocol does not implement a `minimum_amount_out` like on curve, and instead uses this `slippage_tolerance` value. This means the `slippage_tolerance` value is crucial to ensure that the depositor is not leaking any value. However, below shown is a situation where if the depositor adds liquidity in certain compositions, they can leak any amount of value.

A POC is run to generate the numbers given here.

Lets say a pool is created and liquidity is provided with `1e6 whale` and `2e6 luna` tokens. It is quite common to have stableswap pools similarly imbalanced, so this is a usual scenario. Now a user deposits `1e4 whale` and `1e5 luna` tokens in this pool.

At the end, the pool composition becomes `1.01e6 whale` and `2.1e6 luna` tokens. The initial liquidity addition created `2997146` lp tokens and the second liquidity addition creates `109702` lp tokens, for a total of `3106848` lp tokens.

These numbers come from running the POC below, which has the output:

```
running 1 test
===Liq addition===
==Balance deltas==
uwhale delta: -1000000
uluna delta : -2000000
lp delta    : 2997146
==Balance deltas==

===Liq addition 2===
==Balance deltas==
uwhale delta: -10000
uluna delta : -100000
lp delta    : 109702
==Balance deltas==
```

So during the slippage check on the second deposit, `pool_sum = 1e6+2e6 + 1e5+1e4 = 3.11e6`, and the `deposit_sum = 1e5+1e4 = 1.1e5`.

```
pool_ratio = 3.11e6/3106848 = 1.001014533
deposit_ratio = 1.1e5/109702 = 1.00271645
```

Now, even if `slippage_tolerance` is set to 0, since `pool_ratio < deposit_ratio`, the transaction goes through. However, the issue is that in the second liquidity addition, the user could have received less than `109702` lp tokens and the transaction would have still gone through.

Say the user receives only `108000` tokens. Then, total pool `lp_tokens = 2997146+108000 = 3105146`

```
pool_ratio = 3.11e6/3105146 = 1.001563212
deposit_ratio = 1.1e5/108000 = 1.018518519
```

This transaction will also pass, since `deposit_ratio > pool_ratio`. However, we can clearly see that the liquidity depositor has lost 1.55% of their deposit. So even with `slippage_tolerance` set to 0, the stableswap pool can accept liquidity that is not in the ratio of the pool liquidity, and depositors can eat large amounts of slippage.

🔗 Proof of Concept

A POC was used to generate the results of the liquidity addition. First, a couple helper functions,

```
fn print_diff(init_bal: [Uint128; 4], final_bal: [Uint128; 4]) -> [i128; 4] {
    let diffs = [
        final_bal[0].u128() as i128 - init_bal[0].u128() as i128,
        final_bal[1].u128() as i128 - init_bal[1].u128() as i128,
```

```

        final_bal[2].u128() as i128 - init_bal[2].u128() as i128,
        final_bal[3].u128() as i128 - init_bal[3].u128() as i128,
    ];

    println!("==Balance deltas==");
    if diffs[0] != 0 {
        println!("uwahle delta: {}", diffs[0]);
    }
    if diffs[1] != 0 {
        println!("uluna delta : {}", diffs[1]);
    }
    if diffs[2] != 0 {
        println!("uusd delta : {}", diffs[2]);
    }
    if diffs[3] != 0 {
        println!("lp delta : {}", diffs[3]);
    }
    println!("==Balance deltas==\n");

    diffs
}

fn calc_state(suite: &mut TestingSuite, creator: &str) -> [Uint128; 4] {
    let uwahle_balance = RefCell::new(Uint128::zero());
    let uluna_balance = RefCell::new(Uint128::zero());
    let uusd_balance = RefCell::new(Uint128::zero());
    let lp_shares = RefCell::new(Uint128::zero());

    suite.query_balance(&creator.to_string(), "uwahle".to_string(), |result| {
        *uwahle_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_balance(&creator.to_string(), "uluna".to_string(), |result| {
        *uluna_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_balance(&creator.to_string(), "uusd".to_string(), |result| {
        *uusd_balance.borrow_mut() = result.unwrap().amount;
    });

    suite.query_all_balances(&creator.to_string(), |balances| {
        for coin in balances.unwrap().iter() {
            if coin.denom.contains("o.wahle.uluna.uusd") {
                *lp_shares.borrow_mut() = coin.amount;
            }
        }
    });

    let uwahle = *uwahle_balance.borrow();
    let uluna = *uluna_balance.borrow();
    let uusd = *uusd_balance.borrow();
    let lp = *lp_shares.borrow();
    [uwahle, uluna, uusd, lp]
}

```

The actual POC to generate the numbers.

► Details

🔗 Recommended mitigation steps

For stableswap, the `slippage_tolerance` should be checked against the `difference` in the price ratios, so `abs(pool_ratio - deposit_ratio)`. This way both sides of the inequality are checked.

jvr0x (MANTRA) confirmed

🔗 [H-11] Stableswap does disjoint swaps, breaking the underlying invariant

Submitted by [carrotsmuggler](#), also found by [Ox1982us](#), [Abdessamed](#), [Abdessamed](#), and [LonnyFlash](#)

[/contracts/pool-manager/src/helpers.rs#L117-L124](#)

[/contracts/pool-manager/src/helpers.rs#L39-L88](#)

🔗 Finding description and impact

In stableswap pools, the invariant that is preserved is a combination of a CPMM and a constant price model. For pools with more than 2 tokens, every token balance is used to compute the invariant.

This is shown in the curve protocol, where the invariant is calculated correctly.

The invariant is made up of two parts, the stable part and the constant product part:

Note: please see scenario in warden's [original submission](#).

This lets every token in the pool stay in parity with the others, and reduces the slippage. The issue is that in the current implementation, instead of summing or taking the product of all the tokens of the pools, the protocol only takes the sum/product of the ask and offer tokens.

For example, in the `compute_swap` function,

```
let new_pool = calculate_stableswap_y(
  n_coins,
  offer_pool,
  ask_pool,
  offer_amount,
  amp,
  ask_precision,
  StableSwapDirection::Simulate,
)?;
```

Only the ask and offer amounts token amounts are sent in. In the internal `calculate_stableswap_y` function, the invariant is calculated using these two only.

```
let pool_sum = match direction {
  StableSwapDirection::Simulate => offer_pool.checked_add(offer_amount)?,
  StableSwapDirection::ReverseSimulate => ask_pool.checked_sub(offer_amount)?,
```

Here's the curve stableswap code for comparison,

```
for _i in range(N_COINS):
  if _i == i:
    _x = x
  elif _i != j:
    _x = xp[_i]
  else:
    continue
  S_ += _x
```

The `sum_invariant` `D` is calculated only with the two tokens in question, ignoring the third or fourth tokens in the pool; while the actual invariant requires a sum of ALL the tokens in the pool. Similarly, calculating in `calculate_stableswap_d` also calculates the sum using only 2 token balances.

```
let sum_pools = offer_pool.checked_add(ask_pool)?;
```

The `n_coins` used in the calculations, however, is correct and equal to the number of tokens in the pool. This is enforced since the reserves length is used, which is set up correctly during pool creation.

```
n_coins: Uint256::from(pool_info.assets.len() as u128),
```

Thus, the `S` and `D` calculated are incorrect. This also influences the outcome of the newton-raphson iterations, since both these quantities are used there.

The result of this is that if a pool has three tokens A, B, C then A-B swaps ignore the liquidity of C. This is because the `S` and `D` calculations will never touch the liquidity of C, since they only deal with the ask and offer tokens.

So for tricrypto pools, the invariant preserved in A-B swaps is different from the invariant preserved in B-C swaps.

The result are swaps with worse slippage profiles. In normal stableswap pools, the pool tries to maintain all the tokens in parity with each other, giving higher slippage if the pool as a whole is imbalanced. So A-B swaps will have lots of slippage if token C is available in a drastically different amount. However, in this case, the pool only cares about the ask and offer tokens, so the slippage will be lower than expected, leading to arbitrage opportunities. This allows the pools to be more manipulatable.

🔗 Proof of Concept

It is evident from the code snippets above that only the `ask` and `offer` token amounts are used for invariant calculations. Other token amounts are not used. The protocol does support pools with 2+ tokens and for those cases, the invariant is incorrect.

🔗 Recommended mitigation steps

Implement the correct invariant for stableswap, by also including the third/other token amounts in the sum and `D` calculations.

jvr0x (MANTRA) confirmed

🔗 [H-12] Pool creators can manipulate the slippage calculation for liquidity providers

Submitted by [DadeKuma](#), also found by [Ox1982us](#)

Pool creation is permissionless, and users can create a pool by specifying asset denoms. The issue is that they can put a different order of the same token denoms, which should result in the same pool, but in fact, it does not.

This ultimately cause the slippage mechanism to use the inverse ratio instead of the correct one, as in other parts of the codebase these values are always ordered, which will cause a loss of funds for the users that provide liquidity as they use the inverted slippage.

🔗 Proof of Concept

Users can create a pool by calling `pool_manager::create_pool` and specifying the `asset_denoms`.

They can call this function with the same denoms, but in a different order. In theory, this should result in the same pool, but this isn't the case.

Suppose Bob adds liquidity on a `[uwahle, uluna]` pool instead of a `[uluna, uwahle]` pool. The slippage tolerance check is triggered:

```
// assert slippage tolerance
helpers::assert_slippage_tolerance(
    &slippage_tolerance,
    &deposits,
    &pool_assets,
    pool.pool_type.clone(),
    share,
    total_share,
);
```

[/contracts/pool-manager/src/liquidity/commands.rs#L271](#)

This results in the following issue: supposing a k-product pool with `n = 2` tokens, we calculate the slippage check:

```
PoolType::ConstantProduct => {
    if deposits.len() != 2 || pools.len() != 2 {
        return Err(ContractError::InvalidPoolAssetsLength {
            expected: 2,
            actual: deposits.len(),
        });
    }
}

//@audit-info added these logs to check the actual ratios
println!("-> Slippage ratios, d1: {}, p1: {}, d2: {}, p2: {}", deposits[0], pools[0], deposits[1], pools[1]);
println!("1st ratio check: {} > {}", Decimal256::from_ratio(deposits[0], deposits[1]) * one_minus_slippage_tolerance,
    Decimal256::from_ratio(pools[0], pools[1]) * one_minus_slippage_tolerance);
println!("2nd ratio check: {} > {}", Decimal256::from_ratio(deposits[1], deposits[0]) * one_minus_slippage_tolerance,
    Decimal256::from_ratio(pools[1], pools[0]) * one_minus_slippage_tolerance);
```

```

->         > Decimal256::from_ratio(pools[0], pools[1])
           || Decimal256::from_ratio(deposits[1], deposits[0])
           * one_minus_slippage_tolerance
->         > Decimal256::from_ratio(pools[1], pools[0])
           {
               return Err(ContractError::MaxSlippageAssertion);
           }
       }
    }
}

```

</contracts/pool-manager/src/helpers.rs#L452>

`deposits` are always [sorted](#) but the pool order is determined on creation. As the pool has `[uwahle, uluna]` instead of `[uluna, uwahle]` denominations, the slippage checks are inverted.

🔗 Coded POC

Run the following test in `contracts/pool-manager/src/tests/integration_tests.rs`:

► Details

Output:

```

-----> Slippage ratios, d1: 1000000, p1: 100000, d2: 120000, p2: 1000000
1st ratio check: 3.3333333333333333 > 0.1
2nd ratio check: 0.048 > 10
thread 'tests::integration_tests::provide_liquidity::audit_test_wrong_slippage_2_kproduct' panicked at cont
called 'Result::unwrap()' on an 'Err' value: Error executing WasmMsg:
  sender: mantra15n2dapfyf7mzz70y0srycnduw5skp0s9u9g74e
  Execute { contract_addr: "mantra1zwv6feuzhy6a9wekh96cd57lsarmqlwxdypdsplw6zhfncqw6ftqlydlr9", msg: {"pro

```

If we switch the `asset_infos` order while creating the pool:

```

let asset_infos = vec![
-   "uwahle".to_string(),
  "uluna".to_string(),
+   "uwahle".to_string(),
];

```

Output:

```

-----> Slippage ratios, d1: 1000000, p1: 1000000, d2: 120000, p2: 100000
1st ratio check: 3.3333333333333333 > 10
2nd ratio check: 0.048 > 0.1
Event { ty: "execute", attributes: [Attribute { key: "_contract_address", value: "mantra1zwv6feuzhy6a9wekh9
Event { ty: "wasm", attributes: [Attribute { key: "_contract_address", value: "mantra1zwv6feuzhy6a9wekh96cd

```

🔗 Recommended mitigation steps

In `pool_manager::create_pool`, consider sorting the asset denoms, similarly to other parts of the code, by introducing a new struct to encapsulate both `asset_denoms` and `asset_decimals` (as they are tied together) and reorder it before creating the pool.

jvrOx (MANTRA) confirmed

🔗 Medium Risk Findings (19)

🔗 [M-01] In edge cases, `create_pool` can either be reverted or allow user underpay fees

Submitted by [oakcobalt](#), also found by [Ox1982us](#), [Abdessamed](#), and [Lambda](#)

</contracts/pool-manager/src/helpers.rs#L563-L564>

🔗 Finding description and impact

`create_pool` is permissionless. User will need to pay both pool creation fee and token factory fee (fees

charged for creating lp token) when creating a pool.

The vulnerabilities are:

1. In `validate_fees_are_paid`, a strict equality check is used between total paid fees in `pool_creation` token and required pool creation fee. `paid_pool_fee_amount == pool_creation_fee.amount`.
2. `denom_creation_fee.iter()` uses `.any` instead of `.all`, which allows user to only pay one coin from `denom_creation`.

```
//contracts/pool-manager/src/helpers.rs
pub fn validate_fees_are_paid(
    pool_creation_fee: &Coin,
    denom_creation_fee: Vec<Coin>,
    info: &MessageInfo,
) -> Result<Vec<Coin>, ContractError> {
    ...
    // Check if the pool fee denom is found in the vector of the token factory possible fee denoms
    if let Some(tf_fee) = denom_creation_fee
        .iter()
        .find(|fee| &fee.denom == pool_fee_denom)
    {
        // If the token factory fee has only one option, check if the user paid the sum of the fees
        if denom_creation_fee.len() == 1usize {
            ...
        } else {
            // If the token factory fee has multiple options besides pool_fee_denom, check if the user paid
            let paid_pool_fee_amount = get_paid_pool_fee_amount(info, pool_fee_denom?);
            //@audit (1) strict equality check. When user is also required to pay denom_creation_fee in pool
            ensure!(
                paid_pool_fee_amount == pool_creation_fee.amount,
                ContractError::InvalidPoolCreationFee {
                    amount: paid_pool_fee_amount,
                    expected: pool_creation_fee.amount,
                }
            );
            ...
        }

        // Check if the user paid the token factory fee in any other of the allowed denoms
        //@audit (2) iter().any() only requires one of denom_creation_fee token to be paid.
        let tf_fee_paid = denom_creation_fee.iter().any(|fee| {
            let paid_fee_amount = info
                .funds
                .iter()
                .filter(|fund| fund.denom == fee.denom)
                .map(|fund| fund.amount)
                .try_fold(Uint128::zero(), |acc, amount| acc.checked_add(amount))
                .unwrap_or(Uint128::zero());

            total_fees.push(Coin {
                denom: fee.denom.clone(),
                amount: paid_fee_amount,
            });

            paid_fee_amount == fee.amount
        });
        ...
    }
}
```

[/contracts/pool-manager/src/helpers.rs#L577](#)

Based on cosmwasm tokenfactory, denom creation fee (`std.coins`) can contain multiple coins and every coin needs to be paid.

```
//x/tokenfactory/simulation/operations.go
func SimulateMsgCreateDenom(tfKeeper TokenfactoryKeeper, ak types.AccountKeeper, bk BankKeeper) simtypes.Op
...
    // Check if sims account enough create fee
    createFee := tfKeeper.GetParams(ctx).DenomCreationFee
    balances := bk.GetAllBalances(ctx, simAccount.Address)
    |>    _, hasNeg := balances.SafeSub(createFee) //@audit-info all denom creation fee tokens have t
    if hasNeg {
        return simtypes.NoOpMsg(types.ModuleName, types.MsgCreateDenom{}.Type(), "Creator r
    }
    ...
}
```

<https://github.com/CosmWasm/token-factory/blob/47dc2d5ae36980bcc03cf746580f7cb3deabc39e/x/tokenfactory/simulation/operations.go#L359-L361>

Flows: `contracts/pool-manager/src/manager/commands::create_pool -> validate_fees_are_paid()`

🔗 Impact

User can either underpay fees, or `create_pool` tx will revert.

🔗 Proof of Concept

Because pool creation fee and all denom creation fee tokens need to be paid. Consider this edge case: A. There are more than one token factory fee tokens required in `denom_creation_fee`. B. One of the *denom creation fee token is the pool creation fee token* (`fee.denom == poolcreation_fee.denom`).

In this case, user is required to send `=` pool token amount (`pool_creation_fee + denom_creation_fee`) and all other denom creation token amount.

If user sends all the required fees, `paid_pool_fee_amount == pool_creation_fee.amount` check will fail because `paid_pool_fee.amount > pool_creation_fee.amount`. This reverts `create_pool`.

If user chooses to take advantage of `denom_creation_fee.iter().any`, user sent `=` `pool_creation_fee +` one of denom creation token amounts (not the pool token). This passes both the strict equality check and `.any`. However, the user only paid `pool_creation_fee` and underpaid `denom_creation_fee`.

🔗 Recommended mitigation steps

1. Change `.any()` -> `.all()`.
2. Because this branch `denom_creation_fee` contains the `pool_creation` token, needs to add a control flow to handle the iteration of `pool_creation` token to check `paid_fee_amount == fee.amount + pool_creation_fee.amount`.

[3docSec \(judge\) commented:](#)

Looks to be intended behavior, as per comment L576.

[jvrOx \(MANTRA\) disputed and commented:](#)

`pool_creation_fee` is the fee for creating the pool while the vec.
`denom_creation_fee` are the tokens that the user can pay for the token factory in. Only 1 is enough, no need to pay in all the denoms listed there.

[3docSec \(judge\) commented:](#)

Behavior is inconsistent with the MantraChain tokenfactory that collects all fees; okay for valid Medium.

🔗 [M-02] Penalty fees can be shared among future farms or expired farms, risks of exploits

Submitted by [oakcobalt](#), also found by [OxAlix2](#), [Oxlookman](#), [Bauchibred](#), [Egis Security](#), [gegul](#), [jasonxiale](#), [Lambda](#), and [Tigerfrake](#)

The penalty fee is a percentage of the value of existing positions that are emergency withdrawn (`withdraw_position`). It is shared equally among farm owners whose farms have the same `lp_denom` as the position.

The vulnerability is that the penalty fee is divided amongst all farms regardless of whether the farms are current, in the future, or already expired, which allows malicious farm owners to exploit.

🔗 Proof of Concept

We see in `withdraw_position`, all farms with the same `lp_denom` are fetched to get farm owners to share penalty fees. And `get_farms_by_lp_denom` fetches all farms indexed by `lp_denom` regardless whether the farm is current, in the future or already expired.

```
//contracts/farm-manager/src/farm/commands.rs

pub(crate) fn withdraw_position(
    ...
) -> Result<Response, ContractError> {
    ...
    //@@audit This gets all farms including future farms or expired farms
    |> let farms = get_farms_by_lp_denom(
        deps.storage,
        &position.lp_asset.denom,
        None,
        Some(MAX_ITEMS_LIMIT),
    );
    // get unique farm owners for this lp denom
    let unique_farm_owners: Vec<Addr> = farms
        .iter()
        .map(|farm| farm.owner.clone())
        .collect::<HashSet<_>>()
        .into_iter()
        .collect();
    ...
    //@@audit penalty fees can be divided among future farms or expired farms.
    let penalty_fee_share_per_farm_owner = Decimal::from_ratio(
        owner_penalty_fee_comission,
        unique_farm_owners.len() as u128,
    )
    .to_uint_floor();
```

[/contracts/farm-manager/src/position/commands.rs#L366-L368](#)

```
//contracts/farm-manager/src/state.rs
pub fn get_farms_by_lp_denom(
    storage: &dyn Storage,
    lp_denom: &str,
    start_after: Option<String>,
    limit: Option<u32>,
) -> StdResult<Vec<Farm>> {
    let limit = limit.unwrap_or(DEFAULT_LIMIT).min(MAX_ITEMS_LIMIT) as usize;
    let start = cw_utils::calc_range_start_string(start_after).map(Bound::ExclusiveRaw);
    FARMS
        .idx
        .lp_denom
        .prefix(lp_denom.to_owned())
        .range(storage, start, None, Order::Ascending)
        .take(limit)
        .map(|item| {
            let (_, farm) = item?;

            Ok(farm)
        })
        .collect()
}
```

[/contracts/farm-manager/src/state.rs#L124](#)

Suppose a malicious farm creator creates a future farm with valid assets and a popular `lp_denom` with one existing farm active. The new farm is valid in the eyes of admin due to the farm has sufficient assets and correct epoch settings.

1. Any penalty fees shares incurred from `withdraw_position`'s emergency feature before the farm start time will be sent to the malicious farm creator.
2. In addition, the malicious farm creator can also target existing farms with more frequent emergency withdraws either due to bugs or market conditions.
3. Before the new farm epoch starts, the malicious farm creator made a profit ($\frac{1}{2} * \text{total owner penalty fees} - \text{a flat pool creation fee} > 0$). The malicious farm creator `close_farm`. All deposited farm assets are transferred back to the malicious creator.

🔗 Impact

Active farm owners get less penalty fee shares due to fee shared among expired farms and future farms. A malicious farm creator could also take penalty fee shares without having to contribute to rewarding.

🔗 Recommended mitigation steps

In `withdraw_position`, before dividing the `owner_penalty_fee_comission`, filter out farms that starts in a future epoch or have expired.

[jvrOx \(MANTRA\) confirmed and commented:](#)

The farm creation fee is going to be set high enough to prevent unserious players to create farms. Additionally, the contract owner can at any point close farms deemed as spam, malicious or dishonest.

However, the recommendation is valid, will likely adopt it.

[3docSec \(judge\) commented:](#)

Shares in penalty fees can, in my understanding, still compensate for farm creation fees, even though with low likelihood; so Medium seems appropriate.

🔗 [M-03] User is unable to claim their reward for the expanded epochs if farm is expanded

Submitted by [OxRajkumar](#), also found by [Oxlookman](#) and [carrotsmuggler](#)

[/contracts/farm-manager/src/manager/commands.rs#L240-L243](#)

🔗 Finding description and Impact

We have a `claim` function in the Farm Manager, which is used to claim rewards if the user has any open positions. This function updates the `LAST_CLAIMED_EPOCH` to the user's last claimed epoch.

Additionally, we have the `expand_farm` function, which is used to expand farms. Technically, the farm creator can expand the farm even after the farm's last reward epoch has been completed.

Let's explore why below:

```
pub(crate) fn is_farm_expired(
    farm: &Farm,
    deps: Deps,
    env: &Env,
    config: &Config,
) -> Result<bool, ContractError> {
    let epoch_response: EpochResponse = deps
        .querier
        // query preliminary_end_epoch + 1 because the farm is preliminary ending at that epoch, including it.
        .query_wasm_smart(
            config.epoch_manager_addr.to_string(),
```

```

    &QueryMsg::Epoch {
        id: farm.preliminary_end_epoch + 1u64,
    },
    )?;

    let farm_ending_at = epoch_response.epoch.start_time;

    Ok(
        farm.farm_asset.amount.saturating_sub(farm.claimed_amount) == Uint128::zero()
        || farm_ending_at.plus_seconds(config.farm_expiration_time) < env.block.time,
    )
}

```

Let's say our starting epoch was 1 and the `preliminary_end_epoch` was 2, meaning the farm was intended only for epoch 1. However, due to the condition `farm_ending_at.plus_seconds(config.farm_expiration_time) < env.block.time`, the farm owner can expand the farm during `epoch 2 + farm_expiration_time` as well.

Now let's say even if farm expanding the farm for 2 epoch, then users who have an open position for that epoch should be able to claim reward, but this is not the case. Let's see how.

Let's say a user has an opened position claims rewards during epoch 2, then he will be able to claim reward from farm for epoch 1 only because `preliminary_end_epoch` is 2 as you can see this in this function.

```

fn compute_farm_emissions(
    farm: &Farm,
    start_from_epoch: &EpochId,
    current_epoch_id: &EpochId,
) -> Result<(HashMap<EpochId, Uint128>, EpochId), ContractError> {
    let mut farm_emissions = HashMap::new();

    let until_epoch = if farm.preliminary_end_epoch <= *current_epoch_id {
        // the preliminary_end_epoch is not inclusive, so we subtract 1
        farm.preliminary_end_epoch - 1u64
    } else {
        *current_epoch_id
    };

    for epoch in *start_from_epoch..until_epoch {
        farm_emissions.insert(epoch, farm.emission_rate);
    }

    Ok((farm_emissions, until_epoch))
}

```

Whenever a user claims a reward, we maintain the `LAST_CLAIMED_EPOCH`. If the user tries to claim again, they will only be able to claim rewards starting from `LAST_CLAIMED_EPOCH + 1`.

There is a possibility that if a user claims rewards during the `preliminary_end_epoch`, and immediately after, the farm owner expands the farm, the user will not be able to claim rewards for the expanded epoch. This issue can occur for up to a maximum of two epochs.

Let's consider a scenario: the farm's `start_epoch` is 1, and the `preliminary_end_epoch` is 2, meaning the user can currently claim rewards only for epoch 1.

Now, the user claims their reward during epoch 2. However, since the `preliminary_end_epoch` is still 2, the user can only claim up to epoch 1. Immediately after the user's claim transaction, the farm owner expands the farm during the same epoch (epoch 2) for 1 additional epoch. This expansion updates the `preliminary_end_epoch` to 3.

Even though the user has an open position for epoch 2, they will not be able to claim the reward for epoch 2. This is because their `LAST_CLAIMED_EPOCH` is now set to 2, and they can only claim rewards starting from epoch 3. However, the user should still be able to claim rewards for epoch 2 as they had an active position during that time.

In the example above, we observed that the user is unable to claim rewards for one epoch, even though they had an open position for that epoch. This issue can extend to an additional epoch if the user claims rewards during the period between `farm_ending_at` and `farm_ending_at.plus_seconds(config.farm_expiration_time)`, and the farm owner expands the farm within the same time frame but after the user's claim transaction.

🔗 Impact

The impact is High because the user will not be able to claim their full reward.

🔗 Proof of Concept

For the proof of concept, let's consider the following scenario:

1. The farm's `start_epoch` is 1, `preliminary_epoch` is 2, `farm_expiration_time` is 1 hour, and each epoch lasts 1 day.
2. A user have a position for epoch 1 with some weight.
3. During epoch 2, the user claims their reward. Due to the `preliminary_epoch` being 2, they can only claim rewards for epoch 1. After this, their `last_claimed_epoch` is updated to 2.
4. In the same epoch (epoch 2), the farm owner submits a transaction after the user's claim and expands the farm for 1 additional epoch, updating the `preliminary_epoch` to 3.
5. The user comes back in epoch 3 to claim their rewards but will not be able to claim for epoch 2, as their `last_claimed_epoch` is still 2. Consequently, they miss the reward for epoch 2.

This illustrates how the issue prevents the user from receiving rewards for epoch 2.

🔗 Recommended mitigation steps

We can mitigate this issue by only allowing farm expansion before `preliminary_epoch - 1` only.

jvr0x (MANTRA) confirmed

[3docSec \(judge\) commented:](#)

| [S-387](#) proposes an alternative solution.

🔗 [M-04] `withdraw_liquidity` lacks slippage protection

Submitted by [Abdessamed](#), also found by [Ox1982us](#), [OxAlix2](#), [OxRajkumar](#), [Bauchibred](#), [carrotsmuggler](#), [DadeKuma](#), [Egis Security](#), [honey-k12](#), [jasonxiale](#), [Sparrow](#), and [Usagi](#)

[/contracts/pool-manager/src/liquidity/commands.rs#L412-L503](#)

🔗 Vulnerability Details

The `withdraw_liquidity` function allows users to withdraw assets from a pool in exchange for burning their LP tokens. However, the function does not provide a mechanism for users to specify the minimum amount of tokens they are willing to accept upon withdrawal. This omission exposes users to the risk of receiving fewer tokens than expected due to market conditions especially for `ConstantProduct` pools between the transaction initiation and execution.

```
pub fn withdraw_liquidity(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    pool_identifier: String,
) -> Result<Response, ContractError> {
    // --SNIP
    let refund_assets: Vec<Coin> = pool
        .assets
        .iter()
        .map(|pool_asset| {
            Ok(Coin {
                denom: pool_asset.denom.clone(),
                amount: Uint128::try_from(
                    Decimal256::from_ratio(pool_asset.amount, Uint256::one())
                        .checked_mul(share_ratio)?
                        .to_uint_floor(),
                ),
            })
        })
        .collect();
```

```

        )?,
    })
})
.collect::<Result<Vec<Coin>, ContractError>>()?
.into_iter()
// filter out assets with zero amount
.filter(|coin| coin.amount > Uint128::zero())
.collect();

let mut messages: Vec<CosmosMsg> = vec![];

// Transfer the refund assets to the sender
messages.push(CosmosMsg::Bank(BankMsg::Send {
    to_address: info.sender.to_string(),
    amount: refund_assets.clone(),
}));

// --SNIP
}

```

As seen above, the function directly calculates the refund amounts and transfers them to the user without any check for slippage or allowing the user to specify a minimum acceptable amount.

🔗 Impact

Users withdrawing their liquidity can receive less amount than they expected.

🔗 Mitigations

Consider allowing users to provide minimum amount of tokens to receive.

[jvr0x \(MANTRA\) disputed and commented:](#)

Users, especially in xyk pools will face impermanent loss when providing liquidity into the pool. That's why there are swap fees going to LPers, to compensate in a way for that potential loss. While having slippage protection in the withdrawal function can help, it would prevent users going out of the pool if the minimum received tokens don't match their expectation.

Will consider it as potential improvement though. This is low, not a medium issue.

[3docSec \(judge\) commented:](#)

Impermanent loss is implicit, but it's reasonable to expect a protection during high volatility - the industry standard of XYK pools (uniswap v2) does [allow users to provide](#) `amountAMin` and `amountBMin`; they are free to set them to 0 if they want the withdrawal to always succeed.

🔗 [M-05] Insufficient check on asset decimals input in `create_pool` allows malicious pool to be created with invalid swap results

Submitted by [oakcobalt](#)

`create_pool` is permissionless and `asset decimals` are inputs from pool creators.

The vulnerability is there are insufficient check on `asset decimals` are valid. If a pool is created with incorrect `asset decimals`, stableswap will use incorrect decimals to scale assets, resulting in invalid swap results.

🔗 Proof of Concept

```

//contracts/pool-manager/src/manager/commands.rs

pub fn create_pool(
    deps: DepsMut,
    env: Env,

```

```

        info: MessageInfo,
        asset_denoms: Vec<String>,
    |> asset_decimals: Vec<u8>,
        pool_fees: PoolFee,
        pool_type: PoolType,
        pool_identifier: Option<String>,
    ) -> Result<Response, ContractError> {
    ...
        //@audit only check on asset_decimals is the array length.
        ensure!(
            !asset_denoms.is_empty()
            && asset_denoms.len() >= MIN_ASSETS_PER_POOL
            && asset_denoms.len() == asset_decimals.len(),
            ContractError::AssetMismatch
        );
    ...
        POOLS.save(
            deps.storage,
            &identifier,
            &PoolInfo {
                pool_identifier: identifier.clone(),
                asset_denoms,
                pool_type: pool_type.clone(),
                lp_denom: lp_asset.clone(),
    |> asset_decimals, //@audit incorrect decimals will be directly saved and used in stableswap.
                pool_fees,
                assets,
            },
        );
    };
}

```

[/contracts/pool-manager/src/manager/commands.rs#L196](#)

We see that in `compute_swap`. If the asset decimals are incorrect, `offer_pool`, `ask_pool` and `offer_amount` will be scaled to incorrect value before being used in `calculate_stableswap_y`, resulting in incorrect swap result.

```

//contracts/pool-manager/src/helpers.rs
pub fn compute_swap(
    n_coins: Uint256,
    offer_pool: Uint128,
    ask_pool: Uint128,
    offer_amount: Uint128,
    pool_fees: PoolFee,
    swap_type: &PoolType,
    offer_precision: u8,
    ask_precision: u8,
) -> Result<SwapComputation, ContractError> {
    ...
    PoolType::StableSwap { amp } => {
    |> let offer_pool = Decimal256::decimal_with_precision(offer_pool, offer_precision)?;
    |> let ask_pool = Decimal256::decimal_with_precision(ask_pool, ask_precision)?;
    |> let offer_amount = Decimal256::decimal_with_precision(offer_amount, offer_precision)?;

        let new_pool = calculate_stableswap_y(
            n_coins,
            offer_pool,
            ask_pool,
            offer_amount,
            amp,
            ask_precision,
            StableSwapDirection::Simulate,
        );
    };
}

```

[/contracts/pool-manager/src/helpers.rs#L196-L198](#)

🔗 Recommended mitigation steps

I don't see a direct way to query asset decimals during `create_pool` for verification. However, the protocol can implement a registry contract with trusted token metadata info, such that `create_pool` can query the registry contract to validate asset decimals are correct atomically.

[jvr0x \(MANTRA\) acknowledged and commented:](#)

Valid point, but that solution was already thought through. If there was such an asset registry contract, it would need to be gated, and then the pool manager wouldn't really be permissionless.

[a_kalout \(warden\) commented:](#)

@3docSec - I agree that this is valid, but I respectfully believe it should be low/QA at best.

Ok, a malicious user can create a pool with messed-up decimals, but there's nothing that obliges users to use that pool. If only one pool was allowed for X denoms, ok, that would be a medium severity issue as users who want to swap these denoms are obligated to use that pool. Users can provide any pool identifier they want when swapping or when providing/withdrawing liquidity. If a user used that "malicious" pool identifier then that's a user error!

[carrotsmuggler \(warden\) commented:](#)

This should be marked low/QA. The reason is that the exploit hinges on users adding liquidity to misconfigured pools. The data is on-chain for all to see, so sophisticated users can just check if the decimals are configured correctly.

Also, if the pool decimals are configured differently, it will be obvious when simulating swaps or liquidity addition, which is mitigated by slippage. So if a user accepts a transaction with very high slippage where they swap 10 usdc for 1 usdt or something, that's on them.

[oakcobalt \(warden\) commented:](#)

@carrotsmuggler - for a stableswap pool, liquidity addition and its slippage calculation rely on correct assets decimals for correct results. If a stableswap pool has incorrect asset decimals, the slippage check for liquidity is also incorrect. In this case, a user cannot rely on slippage protection to prevent loss.

[oakcobalt \(warden\) commented:](#)

1. **Vulnerable case:** `provide_liquidity` in a stableswap pool with incorrect asset decimals will result in incorrect liquidity calculation and incorrect slippage implementation. A user cannot rely on slippage protection to prevent losses.
2. **Attack:** Sophisticated users can sandwich less sophisticated users on a pool with incorrect asset decimals. This also gives incentives for malicious pool creators to profit from pool users.
3. Note that pools with incorrect asset decimals are not correctable. It's different from a faulty initialization pool price that can be arbitrated to normal price.
4. Based on C4 guideline assets can be at risks out of users control, with attack path. I think it should be Medium.

2 — Med: Assets not at direct risk, but the function of the protocol or its availability could be impacted, or leak value with a hypothetical attack path with stated assumptions, but external requirements.

[3docSec \(judge\) commented:](#)

I agree with Medium. It's a non-obvious attack path that can fall through users' due diligence checks, and the ineffectiveness of slippage protection is key here.

🔗 [M-06] Spread calculation does not account for swap fees

Submitted by [Abdessamed](#), also found by [Ox1982us](#), [OxRajkumar](#), and [DOWSERS](#)

[/contracts/pool-manager/src/helpers.rs#L182-L185](#)

🔗 Vulnerability Details

The spread represents the difference between the expected trade amount and the actual trade amount received during a swap. It is calculated to verify that the slippage remains within the user-defined tolerance during the swap operation. The spread formula varies depending on the pool type:

```
pub fn compute_swap(
    n_coins: Uint256,
    offer_pool: Uint128,
    ask_pool: Uint128,
    offer_amount: Uint128,
    pool_fees: PoolFee,
    swap_type: &PoolType,
    offer_precision: u8,
    ask_precision: u8,
) -> Result<SwapComputation, ContractError> {
    // --SNIP
    match swap_type {
        PoolType::ConstantProduct => {
            let return_amount: Uint256 = Decimal256::from_ratio(ask_pool.mul(offer_amount), offer_pool + of
            let exchange_rate = Decimal256::checked_from_ratio(ask_pool, offer_pool).map_err(|_| ContractEr
            let spread_amount: Uint256 = (Decimal256::from_ratio(offer_amount, Uint256::one()).checked_mul(
            @>>> .checked_sub(return_amount)?;
            // --SNIP
            let fees_computation: FeesComputation = compute_fees(pool_fees, return_amount)?;

            Ok(get_swap_computation(
                return_amount,
                spread_amount,
                fees_computation,
            ))
        }
        PoolType::StableSwap { amp } => {
            // --SNIP
            let return_amount = ask_pool.to_uint256_with_precision(u32::from(ask_precision))?.checked_sub(

            // the spread is the loss from 1:1 conversion
            // thus is it the offer_amount - return_amount
            let spread_amount = offer_amount.to_uint256_with_precision(u32::from(ask_precision))?
            @>>> .saturating_sub(return_amount);

            let fees_computation = compute_fees(pool_fees, return_amount)?;

            Ok(get_swap_computation(
                return_amount,
                spread_amount,
                fees_computation,
            ))
        }
    }
}
```

The issue, as highlighted above, is the fact that the spread is calculated based on the `return_amount` without excluding the fees. As a result, the calculated `spread_amount` is higher than intended in which the slippage tolerance [check](#) in [assert_max_spread](#) may pass, when it should not.

🔗 Impact

The calculated `spread_amount` is higher than the actual slippage because it includes the swap fees.

🔗 Recommended mitigation steps

Consider subtracting the fees from `return_amount` before calculating the `spread_amount`.

jvr0x (MANTRA) confirmed

🔗 [M-07] `query_reverse_simulation` doesn't account for extra fees when simulating stable reversed swaps

Submitted by [OxAlix2](#)

[/contracts/pool-manager/src/queries.rs#L117-L120](#)

🔗 Finding description and impact

Mantra only allows exact-in swaps; however, it provides a way for traders to compute a required amount to get an exact amount out. This could be done using `query_reverse_simulation`, from the code comments:

```
// Queries a swap reverse simulation. Used to derive the number of source tokens returned for  
// the number of target tokens.
```

`query_reverse_simulation` considers all the variable factors that affect the amounts in and out, most importantly fees. However, the issue is that for stable swaps it doesn't consider extra fees, it just account swap, protocol, and burn fees:

```
let before_fees = (Decimal256::one()  
    .checked_sub(pool_fees.protocol_fee.to_decimal_256())?  
    .checked_sub(pool_fees.swap_fee.to_decimal_256())?  
    .checked_sub(pool_fees.burn_fee.to_decimal_256())?  
    .inv()  
    .unwrap_or_else(Decimal256::one)  
    .checked_mul(Decimal256::decimal_with_precision(  
        ask_asset.amount,  
        ask_decimal,  
    ))?);
```

This leads to a wrong `offer_amount` returned, leading to a wrong ask `return_amount`, ultimately leading to unexpected results for traders and unexpected reverts to contracts built on top of this.

🔗 Proof of Concept

Add the following test in `contracts/pool-manager/src/tests/integration_tests.rs`:

► Details

🔗 Recommended mitigation steps

Make sure extra fees are accounted for in the `before_fees` calculation:

```

let before_fees = (Decimal256::one()
  .checked_sub(pool_fees.protocol_fee.to_decimal_256())?
  .checked_sub(pool_fees.swap_fee.to_decimal_256())?
  .checked_sub(pool_fees.burn_fee.to_decimal_256())?
  .checked_sub(pool_fees.extra_fees.iter().fold(
    Decimal256::zero(),
    |acc, fee| {
      acc.checked_add(fee.to_decimal_256())
        .unwrap_or(Decimal256::zero())
    },
  ))?)
  .inv()
  .unwrap_or_else(Decimal256::one)
  .checked_mul(Decimal256::decimal_with_precision(
    ask_asset.amount,
    ask_decimal,
  ))?);

```

[3docSec \(judge\) commented:](#)

Looks valid, can be intended behavior though.

[jvr0x \(MANTRA\) disputed and commented:](#)

This is a valid issue; however, the extra fees were quickly added in one of the subsequent commits after the v1.0.0 tag was done.

This is how the current code on chain looks like:

```

let mut extra_fees = Decimal256::zero();
for extra_fee in pool_fees.extra_fees.iter() {
  extra_fees = extra_fees.checked_add(extra_fee.to_decimal_256())?;
}

let before_fees = (Decimal256::one()
  .checked_sub(pool_fees.protocol_fee.to_decimal_256())?
  .checked_sub(pool_fees.swap_fee.to_decimal_256())?
  .checked_sub(pool_fees.burn_fee.to_decimal_256())?
  .checked_sub(extra_fees)?
  .inv()
  .unwrap_or_else(Decimal256::one)
  .checked_mul(Decimal256::decimal_with_precision(
    ask_asset.amount,
    ask_decimal,
  ))?);

```

[3docSec \(judge\) commented:](#)

I consider this valid because we have to base on the commit where scope was frozen.

🔗 **[M-08] `_compute_offer_amount` floors the `offer_amount` when simulating constant product reversed swaps, leading to unexpected results**

Submitted by [OxAlix2](#)

[/contracts/pool-manager/src/helpers.rs#L355-L364](#)

🔗 Finding description and impact

Mantra only allows exact-in swaps; however, it provides a way for traders to compute a required amount to get an exact amount out. This could be done using `query_reverse_simulation`, from the code comments:

```
// Queries a swap reverse simulation. Used to derive the number of source tokens returned for  
// the number of target tokens.
```

`query_reverse_simulation` calls `compute_offer_amount`, which computes `offer_amount` but floors the result:

```
let cp: Uint256 = offer_asset_in_pool * ask_asset_in_pool;  
let offer_amount: Uint256 = Uint256::one()  
  .multiply_ratio(  
    cp,  
    ask_asset_in_pool.checked_sub(  
      Decimal256::from_ratio(ask_amount, Uint256::one())  
        .checked_mul(inv_one_minus_commission)?  
        .to_uint_floor(),  
    )?,  
  )  
  .checked_sub(offer_asset_in_pool)?;
```

The calculation here differs a bit from the calculation in `compute_swap`, this results in some dust differences. The main issue in the above is that it rounds the result down, which could result in underpayment, ultimately leading to unexpected results for traders and unexpected reverts to contracts built on top of this.

NB: It is better to pay some extra dust amount and get an expected amount out, rather than paying dust amount less and getting an amount less than expected.

🔗 Proof of Concept

Add the following test in `contracts/pool-manager/src/tests/integration_tests.rs`:

► Details

🔗 Recommended mitigation steps

Round the `offer_amount` up, to make sure that the user is receiving the expected amount:

```
pub fn compute_offer_amount(  
  offer_asset_in_pool: Uint128,  
  ask_asset_in_pool: Uint128,  
  ask_amount: Uint128,  
  pool_fees: PoolFee,  
) -> StdResult<OfferAmountComputation> {  
  // ...  
  
  let offer_amount: Uint256 = Uint256::one()  
    .multiply_ratio(  
      cp,  
      ask_asset_in_pool.checked_sub(  
        Decimal256::from_ratio(ask_amount, Uint256::one())  
          .checked_mul(inv_one_minus_commission)?  
          .to_uint_floor(),  
      )?,  
    )  
    .checked_sub(Uint256::one())?,  
  )  
  .checked_sub(offer_asset_in_pool)?;  
  
  // ...  
}
```

[jvrOx \(MANTRA\) acknowledged and commented:](#)

```
Known issue. However, to convert from Decimal to Uint there's a sacrifice to pay, you can either floor  
or round up. Rounding up, what you suggest with paying some more dust, could lead to the contract  
leaking value and eventually leak value from pools which should not happen.
```

[3docSec \(judge\) decreased severity to Low and commented:](#)

Marked low because the impact is limited to dust.

[a kalout \(warden\) commented:](#)

@jvr0x - I agree that there's a tradeoff here; however, we have 2 options:

1. Round down, and return wrong results to the user. For example, I want X amount of T1 out, I simulate that swap by calling `query_reverse_simulation`, and I get an amount Y needed of T2. The catch is that swapping Y of T2 for T1 gives me X-Z, which could easily lead to unexpected reverts for swappers, but no dust left in the contract.
2. Roundup, and have the needed amount of T2 as Y+Z (where Z is dust), in that case when I swap Y+Z of T2 in return for T1, I get exactly what I want of T1 X (and maybe some dust), but some dust could end up on the contract.

These are the only 2 options we have here with their tradeoffs, the protocol is going with the first, which I disagree with, as it has worse consequences than the second. I believe the second option is what should be followed in this case.

[jvr0x \(MANTRA\) commented:](#)

Ah, I had misinterpreted your point. Probably better to round up on the reverse simulation to be on the safe side, you are right.

[3docSec \(judge\) increased severity to Medium and commented:](#)

I'd stick with valid Medium here, because the use-case for the very reasonable "exact output" flow is off-by-one.

[\[M-09\] Single sided liquidity can't be used to lock LP tokens in the farm manager](#)

Submitted by [OxAlix2](#), also found by [OxRajkumar](#), [Abdessamed](#), [carrotsmugger](#), [Egis_Security](#), and [Tigerfrake](#)

</contracts/pool-manager/src/liquidity/commands.rs#L282-L286>

[Finding description and impact](#)

When users provide liquidity into different pools, they call the `provide_liquidity` function, they are also allowed to pass `unlocking_duration`, if provided, the minted LP shares are locked in the farm manager for extra rewards, see [here](#). The protocol doesn't allow users to open positions in the farm manager on behalf of other users, from the docs:

Note: It's only possible to lock an LP position in the farm for the same user providing the liquidity, and not do it on behalf of another user.

It gets validated using the following:

```
// check if receiver is the same as the sender of the tx
ensure!(
    receiver == info.sender.to_string(),
    ContractError::Unauthorized
);
```

On the other hand, when providing single-sided liquidity, half of the deposited amount is swapped to the other asset and then deposited, the process is as follows: `provide_liquidity -> swap -> rely -> provide_liquidity`.

However, the issue here is that the second `provide_liquidity` gets called from the contract itself, i.e., `info.sender` is the contract address itself, which forces the above receiver validation to revert wrongly.

This blocks users from depositing single-sided liquidity and locks the LP tokens in the farm manager in a single action, breaking a core functionality.

🔗 Proof of Concept

Add the following test in `contracts/pool-manager/src/tests/integration_tests.rs#locking_lp`:

► Details

🔗 Recommended mitigation steps

Modify the receiver validation in the `if let Some(unlocking_duration) = unlocking_duration` block, to bypass the condition if the sender is the contract itself.

```
ensure!(
    receiver == info.sender.to_string() || info.sender == env.contract.address,
    ContractError::Unauthorized
);
```

To ensure that this doesn't cause any flaws, a receiver validation should be added to the `if is_single_asset_provision` block, to ensure that the sent receiver is valid, is it enough to do the following:

```
ensure!(
    receiver == info.sender.to_string(),
    ContractError::Unauthorized
);
```

jvr0x (MANTRA) confirmed

🔗 [M-10] Protocol fees are mistakenly configured by protocol pools rather than being imposed

Submitted by [Abdessamed](#)

[/contracts/pool-manager/src/manager/commands.rs#L81](#)

🔗 Impact

For every swap operation, the protocol is entitled to a get fees from the tokens out, the sponsor has confirmed that fees should be taken for every swap. However, the current implementation incorrectly gives the `protocol_fee` control to pool creators when creating pool:

```
pub fn create_pool(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    asset_denoms: Vec<String>,
    asset_decimals: Vec<u8>,
    pool_fees: PoolFee,
    pool_type: PoolType,
```

```

        pool_identifier: Option<String>,
    ) -> Result<Response, ContractError> {
        // --SNIP
        POOLS.save(
            deps.storage,
            &identifier,
            &PoolInfo {
                pool_identifier: identifier.clone(),
                asset_denoms,
                pool_type: pool_type.clone(),
                lp_denom: lp_asset.clone(),
                asset_decimals,
                pool_fees,
                assets,
            },
        )?;
    }
}

```

Pool creators can specify 0 fees for `pool_fees.protocol_fee`, causing the protocol to lose swap fees they are entitled to.

🔗 Recommended mitigation steps

Consider moving the `protocol_fee` to the contract's config, rather than being controlled by pool creators.

jvr0x (MANTRA) confirmed

[3docSec \(judge\) commented:](#)

Looks valid, probably Low though, as there is no impact on users.

[DadeKuma \(warden\) commented:](#)

This should be Low/info at most because `pool_fees.protocol_fee` is a "frontend" fee (i.e., a fee charged for using the user interface that facilitates interaction with these contracts). This is common in many protocols.

The main fees, which are also implemented correctly (and fetched from the config), are the pool creation fee and the token factory fee.

[Abdessamed \(warden\) commented:](#)

*@DadeKuma - the fees you are talking about are the **pool creation fees**, which is implemented correctly and the report does not speak about these fees at all.*

*What the report is highlighting is that the **swap fees** are given control to the pool creators mistakenly rather than being imposed by the protocol's configuration whereby pool creators can simply put zero fees for the protocol team to prevent them from taking swap fees for whatever reason. The sponsor intends to take fees from every swap operation to collect revenue, which is not implemented.*

*@3docSec - there is no impact for pool creators but at the **expense of protocol losses**. The protocol team intends to collect revenue from swap fees (like Uniswap and other AMMs do), and not receiving those fees is a clear loss for the protocol team, and thus, this report is eligible for Medium severity.*

[3docSec \(judge\) commented:](#)

@Abdessamed - I agree. I do not think this is a likely attack vector because the protocol can:

- Force through the UI a proper value if creators create pools through the UI.
- Not show in the UI pools that don't have an adequate protocol fee for swap users.

We are a bit borderline here because of likelihood, but Medium seems justified to me.

🔗 [M-11] When a user single-side deposit into a pool, slippage protection is invalid

Submitted by [oakcobalt](#), also found by [Egis Security](#).

When a user single-side deposit into a pool, a swap is performed first before liquidity provision.

The vulnerability is `belief_price` is always set to `NONE` which causes slippage protection to be invalid in some cases.

```
//contracts/pool-manager/src/liquidity/commands.rs

pub fn provide_liquidity(
    ...
) -> Result<Response, ContractError> {
    ...
    let is_single_asset_provision = deposits.len() == 1usize;

    if is_single_asset_provision {
        ...
        Ok(Response::default()
            .add_submessage(SubMsg::reply_on_success(
                wasm_execute(
                    env.contract.address.into_string(),
                    &ExecuteMsg::Swap {
                        ask_asset_denom,
                        belief_price: None,
                        max_spread,
                        receiver: None,
                        pool_identifier,
                    },
                    vec![swap_half],
                ),
                SINGLE_SIDE_LIQUIDITY_PROVISION_REPLY_ID,
            ))
            .add_attributes(vec![("action", "single_side_liquidity_provision"))]))
    } else {
```

[/contracts/pool-manager/src/liquidity/commands.rs#L164](#)

🔗 Proof of Concept

During a swap (`perform_swap::assert_max_spread`), `belief_price` and `max_spread` are both user input and can be combined for slippage protection.

- `belief_price`: what a user believes the spot price before swap should be.
- `max_spread`: how much the price moved during swap.

```
// If `belief_price` and `max_spread` both are given,
// we compute new spread else we just use pool network
// spread to check `max_spread`
pub fn assert_max_spread(
    belief_price: Option<Decimal>,
    max_spread: Option<Decimal>,
    offer_amount: Uint128,
    return_amount: Uint128,
    spread_amount: Uint128,
) -> StdResult<()> {
    let max_spread: Decimal256 = max_spread
        .unwrap_or(Decimal::from_str(DEFAULT_SLIPPAGE)?)
        .min(Decimal::from_str(MAX_ALLOWED_SLIPPAGE)?)
```

```

        .into());
    if let Some(belief_price) = belief_price {
        let expected_return = Decimal::from_ratio(offer_amount, Uint128::one())
            .checked_mul(
                belief_price
                    .inv()
                    .ok_or_else(|| StdError::generic_err("Belief price can't be zero"))?,
            )?
            .to_uint_floor();

        let spread_amount = expected_return.saturating_sub(return_amount);

        if return_amount < expected_return
            && Decimal256::from_ratio(spread_amount, expected_return) > max_spread
        {
            return Err(StdError::generic_err("Spread limit exceeded"));
        }
    } else if Decimal256::from_ratio(spread_amount, return_amount + spread_amount) > max_spread {
        return Err(StdError::generic_err("Spread limit exceeded"));
    }
    Ok(())
}

```

[/contracts/pool-manager/src/swap/perform_swap.rs#L166](#)

`belief_price` is especially important in constant product swap case where the pool spot price can shift wildly between tx submission and tx execution.

Even in stableswap pool, `belief_price` is also important because pool spot price can still shift away from the ideal 1:1 price in the case where `amp` is set to a low value (e.g., less than 2000000) or assets are more imbalanced.

When `belief_price` is set to NONE, `max_spread`'s implementation between constant product swap and stable swap become inconsistent. Both constant product pool and stableswap pool's slippage protection can be invalid.

- Constant product swap (`belief_price = none`)

$$\text{spread_amount} = \text{offer_amount} * (\text{ask_pool} / \text{offer_pool}) - \text{return_amount};$$

Note: `ask_pool / offer_pool` is the current spot price of the pool, which is not foreseeable by the user.

Check: $\text{spread_amount} / (\text{spread_amount} + \text{return_amount}) \leq \text{max_spread}$

- stableswap (`belief_price = none`)

$$\text{spread_amount} = \text{offer_amount} - \text{return_amount}; \text{ (with decimal aligned).}$$

Note: `spread_amount` is based on a [fixed 1:1 price](#). However, when assets are imbalanced or `amp` is low, the $\text{deltaX}:\text{deltaY}$ will differ from 1:1.

We see in both cases, swap slippage protection is voided because it's not able to account for the pool price slipping before tx execution.

🔗 Recommended mitigation steps

Consider revising `provide_liquidity` to allow user to pass a `belief_price` for single-sided swap flow.

[jvr0x \(MANTRA\) confirmed and commented:](#)

Valid, probably low.

[a_kalout \(warden\) commented:](#)

When I, as a user, am providing liquidity to a pool, why would I care about the underlying swap that is

happening behind the scenes? I don't think it would make sense to put "conditions" (slippage) on that swap. What I care about is the result of providing liquidity and the shares minted to me, which are validated using `slippage_tolerance`.

As a result, I believe `slippage_tolerance` is more than enough as slippage protection, even when providing single-side liquidity. I believe this could be a low-severity issue.

[carrotsmuggler \(warden\) commented:](#)

The issue states that slippage is not set, which is incorrect since `max_spread` is used. The issue then says `max_spread` is not good for stableswap and constant products.

`max_spread` not being good is already covered in [F-37](#), so this is not a new path and is basically just triggering a flaw reported in another issue.

The `max_spread` implementation on constant product pool does work. It calculates `ask_pool / offer_pool`, the current spot price of the pool, which is a measure of the amount of price change the user expects. The submitter says that "Both constant product pool and stableswap pool's slippage protection can be invalid.", but does not prove that statement beyond what is already reported in [F-37](#).

Furthermore, the issue here shows that `belief_price` is being ignored and `max_spread` is being used instead.

[oakcobalt \(warden\) commented:](#)

The `max_spread` implementation on constant product pool does work. It calculates `ask_pool / offer_pool` ...

`max_spread` deals with `spread_amount / (return_amount + spread_amount)`. it uses the spot price (`ask_pool / offer_pool`) when `belief_price` is not provided.

the current spot price of the pool, which is a measure of the amount of price change the user expects.

This is not entirely correct. The user cannot measure the price change based on spot price without the `belief_price`, which is the vulnerable case this report deals with.

`max_spread` not being good is already covered in [F-37](#), so this is not a new path and is basically just triggering a flaw reported in another issue.

Not true. [F-37](#) only deals with stableswap case, and it has to do with a comparison logic specific in the stableswap branch. It doesn't consider constant product pool.

[oakcobalt \(warden\) commented:](#)

The point here is not that there is no slippage check for `provide_liquidity`, it's that when `belief_price` is none when single side depositing, existing slippage calculation is invalid.

For a constant product pool:

1. When `belief_price` is none, user cannot control how much ask tokens to transfer out, which means the user cannot control the asset ratio ($\text{ask_token} / \text{ask_pool}$, or $\text{offer_token} / \text{offer_pool}$) which determines actual minted LPs.
2. `max_spread` doesn't prevent slippage without `belief_price`. It doesn't care how much price moved before the swap.
3. `slippage_tolerance` is intended to work when no swaps, i.e., user deposits pool reserve ratio $\rightarrow$ deposit ratio is fixed by user. It only cares about input deposits ratio's deviation from the pool reserve ratio. ($\text{ask_token} / \text{offer_token}$ vs $\text{ask_pool} / \text{offer_pool}$).

However, because of (1), both deposit ratio and pool reserve ratio are moving targets, resulting in invalid `slippage_tolerance`'s comparison.

Example:

- User provides 10000 DAI to a constant product pool with 20% slippage for simplicity.
- Pool's total lp: 316

Case1: Pool's spot price before swap: 5000 DAI, 20 WETH

1. swap: $\rightarrow$ 5000 DAI, 10 WETH

2. two-sided `provide_liquidity`:

- pool reserve: 10000 DAI, 10 WETH
- $\text{deposit}[0] / \text{deposit}[1] = 500$
- $\text{pool}[0] / \text{pool}[1] = 1000$
- $\text{pool}[0] / \text{pool}[1] > \text{deposit}[0] / (\text{deposit}[1] * 80\%) \rightarrow$ slippage check fails.
- Theoretical Lp mints: 158

Case2: Pool's spot price before swap: 20000 DAI, 5 WETH

1. swap: $\rightarrow$ 5000 DAI, 1 WETH

2. two-sided `provide_liquidity`:

- pool reserve: 25000 DAI, 4 WETH
- $\text{deposit}[0] / \text{deposit}[1] = 5000$
- $\text{pool}[0] / \text{pool}[1] = 6250$
- $(\text{deposit}[0] / \text{deposit}[1]) * 80\% \leq \text{pool}[0] / \text{pool}[1] \leq \text{deposit}[0] / (\text{deposit}[1] * 80\%) \rightarrow$ slippage check pass.
- Lp mints: 63

As seen, Case2's slippage check passed; however, actual minted share is much lower compared to Case1.

[M-12] Insufficient intermediate value precision in StableSwap calculations

Submitted by [Lambda](#)

The `calculate_stableswap_y` function uses `Uint256` for intermediate calculations which can cause arithmetic overflow errors when dealing with large token amounts, especially for tokens with high decimal places (e.g. 18 decimals). This significantly impacts the reliability of the StableSwap pool implementation.

Proof of Concept

The issue occurs in [src/helpers.rs#L106-L152](#) where `calculate_stableswap_y` performs multiplication and division operations using `Uint256`.

The problem manifests when performing swaps with large token amounts. A proof of concept that demonstrates the failure is provided here:

```
#[test]
fn test_stableswap_large_swap() {
    // Use realistic large pool values (e.g. 1M tokens with 18 decimals)
    let large_pool = Uint128::new(1_000_000_000_000_000_000_000_000u128); // 1M tokens

    // Large swap amount (100k tokens)
    let large_amount = Uint128::new(100_000_000_000_000_000_000_000u128);

    let test_cases = vec![
        // (amp, offer_pool, ask_pool, offer_amount, description)
        (
            MIN_AMP,
            large_pool,
            large_pool,
            large_amount,
            "Balanced pool, minimum amplification"
        ),
    ];

    for (amp, offer_pool, ask_pool, offer_amount, description) in test_cases {
        println!("Testing case: {}", description);
        println!("Parameters: amp={}, offer_pool={}, ask_pool={}, offer_amount={}",
            amp, offer_pool, ask_pool, offer_amount);

        // Convert to Decimal256 for precision
        let offer_pool_dec = Decimal256::from_ratio(offer_pool, Uint128::new(1));
        let ask_pool_dec = Decimal256::from_ratio(ask_pool, Uint128::new(1));
        let offer_amount_dec = Decimal256::from_ratio(offer_amount, Uint128::new(1));

        let result = calculate_stableswap_y(
            Uint256::from(2u8),
            offer_pool_dec,
            ask_pool_dec,
            offer_amount_dec,
            &amp,
            18, // Using 18 decimals precision
            StableSwapDirection::Simulate,
        );

        match result {
            Ok(value) => {
                println!("Result: {}", value);
            }
            Err(e) => {
                panic!(
                    "Failed to converge: {:?}",
                    e
                );
            }
        }
    }
}
```

The test fails with `CheckedMultiplyRatioError(Overflow)` because intermediate calculations in `calculate_stableswap_y` overflow `Uint256` when dealing with:

1. Pool amounts of 1M tokens with 18 decimals (`1e24`).
2. Swap amounts of 100k tokens with 18 decimals (`1e23`).

These are realistic values that could occur in production, making this an issue that could prevent legitimate swaps from executing. Moreover, the following was mentioned in the audit description:

Approximation errors when handling very large amounts, especially with assets having 18 decimal places. Think of handling trillions of such assets.

🔗 Recommended mitigation steps

Replace `Uint256` with `Uint512` for intermediate calculations in `calculate_stableswap_y`. This provides sufficient precision to handle large token amounts with high decimal places.

```
pub fn calculate_stableswap_y(
  n_coins: Uint256,
  offer_pool: Decimal256,
  ask_pool: Decimal256,
  offer_amount: Decimal256,
  amp: &u64,
  ask_precision: u8,
  direction: StableSwapDirection,
) -> Result<Uint128, ContractError> {
  // Convert inputs to Uint512 for intermediate calculations
  let ann = Uint512::from(Uint256::from_u128((*amp).into()).checked_mul(n_coins?));
  // ... rest of calculations using Uint512
}
```

The final result can still be converted back to `Uint128` since the actual swap amounts will fit within that range. This change ensures the contract can handle realistic token amounts while maintaining precision in the StableSwap calculations.

A test suite should be added that verifies the contract handles large token amounts correctly across different pool configurations and swap scenarios.

jvr0x (MANTRA) confirmed

🔗 [M-13] Wrong simulation function used in reverse operation path

Submitted by [Rhaydden](#), also found by [Tigerfrake](#)

`reverse_simulate_swap_operations` function incorrectly uses `query_simulation` instead of `query_reverse_simulation` when calculating multi-hop trades in reverse. As a result, users will get incorrect price calculations when they attempt to determine how many input tokens they need for a desired output amount.

In summary:

1. Users receive incorrect price quotes for trades.
2. The error compounds in multi-hop trades, causing issues.

🔗 Proof of Concept

</contracts/pool-manager/src/queries.rs#L275>

The bug is in the `reverse_simulate_swap_operations` function:

```
pub fn reverse_simulate_swap_operations(
  deps: Deps,
  ask_amount: Uint128,
  operations: Vec<SwapOperation>,
) -> Result<SimulateSwapOperationsResponse, ContractError> {
  let operations_len = operations.len();
  if operations_len == 0 {
    return Err(ContractError::NoSwapOperationsProvided);
  }

  let mut amount = ask_amount;

  for operation in operations.into_iter().rev() {
    match operation {
      SwapOperation::MantraSwap {
        token_in_denom,
        token_out_denom,
        pool_identifier,
```

```

    } => {
    >>> let res = query_simulation(
        deps,
        coin(amount.u128(), token_out_denom),
        token_in_denom,
        pool_identifier,
    );
    amount = res.return_amount;
    }
}

Ok(SimulateSwapOperationsResponse { amount })
}

```

According to the docs and the example given in it:

We have a QUERY:

```

{
  "reverse_simulation": {
    "ask_asset": {
      "denom": "uusdc",
      "amount": "990000"
    },
    "offer_asset_denom": "uom",
    "pool_identifier": "om-usdc-1"
  }
}

```

And a RESPONSE (ReverseSimulationResponse):

```

{
  "offer_amount": "1000000",
  "spread_amount": "5000",
  "swap_fee_amount": "3000",
  "protocol_fee_amount": "2000",
  "burn_fee_amount": "0",
  "extra_fees_amount": "0"
}

```

The doc also clearly states that offer_amount in the response is “The amount of the offer asset needed to get the ask amount.”

Now, let’s compare again with the implementation:

```

let res = query_simulation( <--- This is WRONG! Its using forward simulation
  deps,
  coin(amount.u128(), token_out_denom),
  token_in_denom,
  pool_identifier,
);

```

The bug is clear because it’s using query_simulation instead of query_reverse_simulation.

Also consider a multi-hop trade A -> B -> C where a user wants 100 C tokens:

1. The fn calculates how many B tokens you get for X C tokens (forward).
2. Then calculates how many A tokens you get for Y B tokens (forward).
3. This is incorrect for reverse price discovery.

It shoould actually be:

1. Calculate how many B tokens you need to get 100 C tokens (reverse).
2. Then calculate how many A tokens you need to get the required B tokens (reverse).

The contract already has a correct `query_reverse_simulation` function that handles proper price calculation for reverse operations, but it's not being used.

🔗 Recommended mitigation steps

```
pub fn reverse_simulate_swap_operations(
  deps: Deps,
  ask_amount: Uint128,
  operations: Vec<SwapOperation>,
) -> Result<SimulateSwapOperationsResponse, ContractError> {
  let operations_len = operations.len();
  if operations_len == 0 {
    return Err(ContractError::NoSwapOperationsProvided);
  }

  let mut amount = ask_amount;

  for operation in operations.into_iter().rev() {
    match operation {
      SwapOperation::MantraSwap {
        token_in_denom,
        token_out_denom,
        pool_identifier,
      } => {
-       let res = query_simulation(
+       let res = query_reverse_simulation(
        deps,
        coin(amount.u128(), token_out_denom),
        token_in_denom,

        pool_identifier,
      )?;
-       amount = res.return_amount;
+       amount = res.offer_amount;
      }
    }
  }

  Ok(SimulateSwapOperationsResponse { amount })
}
```

[jvrOx \(MANTRA\) disputed and commented:](#)

This is something that was fixed in a subsequent commit after the v1.0.0 tag was created. The current (live) reverse query code is the following:

```
pub fn reverse_simulate_swap_operations(
  deps: Deps,
  ask_amount: Uint128,
  operations: Vec<SwapOperation>,
) -> Result<ReverseSimulateSwapOperationsResponse, ContractError> {
  let operations_len = operations.len();
  if operations_len == 0 {
    return Err(ContractError::NoSwapOperationsProvided);
  }

  let mut offer_in_needed = ask_amount;
  let mut spreads: Vec<Coin> = vec![];
  let mut swap_fees: Vec<Coin> = vec![];
  let mut protocol_fees: Vec<Coin> = vec![];
  let mut burn_fees: Vec<Coin> = vec![];
  let mut extra_fees: Vec<Coin> = vec![];

  for operation in operations.into_iter().rev() {
    match operation {
      SwapOperation::MantraSwap {
        token_in_denom,
        token_out_denom,
        pool_identifier,
      } => {
        let res = query_reverse_simulation(
          deps,
          coin(offer_in_needed.u128(), token_out_denom.clone()),
          token_in_denom,
```

```

        pool_identifier,
    )?;

    if res.spread_amount > Uint128::zero() {
        spreads.push(coin(res.spread_amount.u128(), &token_out_denom));
    }
    if res.swap_fee_amount > Uint128::zero() {
        swap_fees.push(coin(res.swap_fee_amount.u128(), &token_out_denom));
    }
    if res.protocol_fee_amount > Uint128::zero() {
        protocol_fees.push(coin(res.protocol_fee_amount.u128(), &token_out_denom));
    }
    if res.burn_fee_amount > Uint128::zero() {
        burn_fees.push(coin(res.burn_fee_amount.u128(), &token_out_denom));
    }
    if res.extra_fees_amount > Uint128::zero() {
        extra_fees.push(coin(res.extra_fees_amount.u128(), &token_out_denom));
    }

    offer_in_needed = res.offer_amount;
}
}

spreads = aggregate_coins(spreads)?;
swap_fees = aggregate_coins(swap_fees)?;
protocol_fees = aggregate_coins(protocol_fees)?;
burn_fees = aggregate_coins(burn_fees)?;
extra_fees = aggregate_coins(extra_fees)?;

Ok(ReverseSimulateSwapOperationsResponse {
    offer_amount: offer_in_needed,
    spreads,
    swap_fees,
    protocol_fees,
    burn_fees,
    extra_fees,
})
}

```

[3docSec \(judge\) commented:](#)

For judging, we base on the commit frozen for the audit scope; so while it's good to see the team found and fixed the issue independently, it wouldn't be fair to warden to not reward a valid finding.

🔗 [M-14] Amplifiers can't be ramped allowing loss of funds from the pool

Submitted by [Bauchibred](#)

The Mantra DEX stableswap implementation lacks the ability to modify the amplification coefficient (A) after pool creation, which is a critical feature present even in the original Curve implementation. While the focus was on reducing Newton-Raphson iterations from 256 to 32, the absence of amplifier ramping breaks the logic.

In Mantra's implementation, the amplification factor is static:

[/packages/amm/src/pool_manager.rs#L85-L94](#)

```

pub enum PoolType {
    StableSwap {
        /// The amount of amplification to perform on the constant product part of the swap formula.
        amp: u64,
    },
    ConstantProduct,
}

```

Once set during pool creation, there is no mechanism to modify this value. In contrast, Curve's implementation includes comprehensive amplifier management, this can be seen [here](#):

```
def ramp_A(_future_A: uint256, _future_time: uint256):
    assert msg.sender == self.owner # dev: only owner
    assert block.timestamp >= self.initial_A_time + MIN_RAMP_TIME
    assert _future_time >= block.timestamp + MIN_RAMP_TIME # dev: insufficient time

    initial_A: uint256 = self._A()
    future_A_p: uint256 = _future_A * A_PRECISION

    assert _future_A > 0 and _future_A < MAX_A
    if future_A_p < initial_A:
        assert future_A_p * MAX_A_CHANGE >= initial_A
    else:
        assert future_A_p <= initial_A * MAX_A_CHANGE

    self.initial_A = initial_A
    self.future_A = future_A_p
    self.initial_A_time = block.timestamp
    self.future_A_time = _future_time
```

Note that the Amplification Coefficient is a crucial feature for managing the pool's behavior and adapting to changing market conditions.

Before going further we understand that the stableswap invariant is enforced by the equation: $A_n \sum x_i + D = AD^n + (D^{n+1}) / (n^n \prod x_i)$, this can be seen in [compute_y_raw\(.\)](#), that gets called from [compute_y\(.\)](#).

Then, this directly translates to how we mint the amount of liquidity say when a user is making a stable swap deposit when providing liquidity:

[/contracts/pool-manager/src/liquidity/commands.rs#L256-L267](#)

```
pub fn provide_liquidity() {

    // ..snip
    // @audit below we route the call when providing the liquidity to the stableswap implementation
    } else {
        compute_lp_mint_amount_for_stableswap_deposit(
            amp_factor,
            // pool_assets hold the balances before the deposit was made
            &pool_assets,
            // add the deposit to the pool_assets to calculate the new balances
            &add_coins(pool_assets.clone(), deposits.clone())?,
            total_share,
        )?
        .ok_or(ContractError::StableLpMintError)?
    }
    // ..snip
}
```

[/contracts/pool-manager/src/helpers.rs#L790-L812](#)

```
pub fn compute_lp_mint_amount_for_stableswap_deposit(
    amp_factor: &u64,
    old_pool_assets: &[Coin],
    new_pool_assets: &[Coin],
    pool_lp_token_total_supply: Uint128,
) -> Result<Option<Uint128>, ContractError> {
    // Initial invariant
    let d_0 = compute_d(amp_factor, old_pool_assets).ok_or(ContractError::StableInvariantError)?;

    // Invariant after change, i.e. after deposit
    // notice that new_pool_assets already added the new deposits to the pool
    let d_1 = compute_d(amp_factor, new_pool_assets).ok_or(ContractError::StableInvariantError)?;

    // If the invariant didn't change, return None
    if d_1 <= d_0 {
        Ok(None)
    } else {
        let amount = Uint512::from(pool_lp_token_total_supply)
            .checked_mul(d_1.checked_sub(d_0)?)?
            .checked_div(d_0)?;
        Ok(Some(Uint128::try_from(amount)?))
    }
}
```

```
}
}
```

That's to say essentially when we are trying to calculate the swap amount, see [here](#):

```
pub fn compute_y_raw(
    n_coins: u8,
    amp_factor: &u64,
    swap_in: Uint128,
    //swap_out: Uint128,
    no_swap: Uint128,
    d: Uint512,
) -> Option<Uint512> {
    let ann = amp_factor.checked_mul(n_coins.into()); // A * n ** n

    // sum' = prod' = x
    // c = D ** (n + 1) / (n ** (2 * n) * prod' * A)
    let mut c = d;

    c = c
        .checked_mul(d)
        .unwrap()
        .checked_div(swap_in.checked_mul(n_coins.into()).unwrap().into())
        .unwrap();

    c = c
        .checked_mul(d)
        .unwrap()
        .checked_div(no_swap.checked_mul(n_coins.into()).unwrap().into())
        .unwrap();

    c = c
        .checked_mul(d)
        .unwrap()
        .checked_div(ann.checked_mul(n_coins.into()).unwrap().into())
        .unwrap();

    // b = sum(swap_in, no_swap) + D // Ann - D
    // not subtracting D here because that could result in a negative.
    let b = d
        .checked_div(ann.into())
        .unwrap()
        .checked_add(swap_in.into())
        .unwrap()
        .checked_add(no_swap.into())
        .unwrap();

    // Solve for y by approximating: y**2 + b*y = c
    let mut y_prev: Uint512;
    let mut y = d;
    for _ in 0..1000 {
        y_prev = y;
        // y = (y * y + c) / (2 * y + b - d);
        let y_numerator = y.checked_mul(y).unwrap().checked_add(c).unwrap();
        let y_denominator = y
            .checked_mul(Uint512::from(2u8))
            .unwrap()
            .checked_add(b)
            .unwrap()
            .checked_sub(d)
            .unwrap();
        y = y_numerator.checked_div(y_denominator).unwrap();
        if y > y_prev {
            if y.checked_sub(y_prev).unwrap() <= Uint512::one() {
                break;
            }
        } else if y_prev.checked_sub(y).unwrap() <= Uint512::one() {
            break;
        }
    }
    Some(y)
}
```

To explain the bug case more, let's examine how different A (Amplifier) values affect a 2-token stableswap pool with DAI and USDC ($n=2$):

Initial Balanced State:

- $x_1 = 1000$ DAI
- $x_2 = 1000$ USDC
- $D = 2000$ (invariant)

With Optimal $A = 85$:, concluding this as optimal considering this was also hinted in the [original Curve implementation](#):

```
Left side:  $An\sum x_i + D$ 
=  $85 * 2 * (1000 + 1000) + 2000$ 
=  $85 * 2 * 2000 + 2000$ 
=  $340,000 + 2000$ 
=  $342,000$ 

Right side:  $AD^n + (D^{n+1})/(n^n \prod x_i)$ 
=  $85 * 2000^2 + 2000^3/(2^2 * 1000 * 1000)$ 
=  $340,000 + 2000$ 
=  $342,000$ 

Price impact for 10% imbalance trade  $\approx 0.3\%$ 
```

With Too High $A = 1000$:

```
Left side:  $An\sum x_i + D$ 
=  $1000 * 2 * (1000 + 1000) + 2000$ 
=  $1000 * 2 * 2000 + 2000$ 
=  $4,000,000 + 2000$ 
=  $4,002,000$ 

Right side:  $AD^n + (D^{n+1})/(n^n \prod x_i)$ 
=  $1000 * 2000^2 + 2000^3/(2^2 * 1000 * 1000)$ 
=  $4,000,000 + 2000$ 
=  $4,002,000$ 

Price impact for 10% imbalance trade  $\approx 0.025\%$ 
```

With Too Low $A = 10$:

```
Left side:  $An\sum x_i + D$ 
=  $10 * 2 * (1000 + 1000) + 2000$ 
=  $10 * 2 * 2000 + 2000$ 
=  $40,000 + 2000$ 
=  $42,000$ 

Right side:  $AD^n + (D^{n+1})/(n^n \prod x_i)$ 
=  $10 * 2000^2 + 2000^3/(2^2 * 1000 * 1000)$ 
=  $40,000 + 2000$ 
=  $42,000$ 

Price impact for 10% imbalance  $\approx 2.5\%$ 
```

This demonstrates how:

1. Optimal A (85) balances stability with safety.
2. Too high A (1000) makes the pool vulnerable to manipulation due to minimal price impact.
3. Too low A (10) causes excessive slippage even for small trades.

🔗 Impact

As already slightly hinted under *Proof of Concept*, we can see how having a static amplification value causes lack of flexibility in managing the pool's behavior, and [would immensely affect the minting lps logic](#) when we are [providing liquidity to a stableswap pool](#), building up on the swap details already hinted:

Note that market conditions are not *constant* and deviate quite, which means we can't have a constant "amp" for all condition of the pool for it's lifetime, just like we can't have say a constant "slippage" for all market conditions. Even at that, assume we end up with an amp value that works 80% of the time, the implementation is still broken for 20% of the time. And blackswan events unfortunately occur from time to

time which would even cause the breach to be way higher.

To build up on the scenario already hinted at, we can see that in the case we have a very high amplifier (A) The curve is flatter, meaning the price *wrongly* remains relatively stable even with significant imbalances. Whereas this is desirable for stablecoins or pegged assets where maintaining a tight peg is paramount, we still have to take into account that a blackswan event could be occurring. As such, the price of one of the assets in the pool is heavily dropping due to some sort of freefall. We can't just keep the price stable or enforce the equilibrium; otherwise, we then allow for anyone with the now "fall dropping" asset to come steal the other assets since the the prices are ~same; i.e., assume when Terra's UST was on a free fall, with a very high Amp value, holders of Terra's UST can come steal the other stablecoined assets in the pool, assume USDC for this report.

For example, during UST's depeg:

- Pool state: 1M UST ($\$0.95$) and 1M USDC ($\1.00).
- With A=1000: A trader could still swap $\sim 950k$ UST for $\sim 950k$ USDC (only $\sim 0.025\%$ price impact).
- With proper ramping, A could be decreased to protect the pool, causing larger price impacts.
- Loss to pool: $\sim \$47,500$ (5% of 950k) due to inability to adjust A.

Now with a low amplifier, the case is even exacerbated, considering the curve is even more curved, leading to even greater price fluctuations with even small imbalances. In this case, we just open up the window for malicious actors to front/back run trades and scoop up the asset that's cheaper in price at the time.

🔗 Recommended Mitigation Steps

Implement amplifier ramping functionality similar to Curve:

```
pub enum ExecuteMsg {
  RampAmplifier {
    future_amp: u64,
    future_time: u64,
  },
  StopRamp {},
}
```

And add safety constraints:

```
const MAX_A: u64 = 1_000_000; // 10^6
const MAX_A_CHANGE: u64 = 10;
const MIN_RAMP_TIME: u64 = 3600; // 1 hour
```

[3docSec \(judge\) commented:](#)

Looks reasonable but more of a missing feature than a bug.

[jvrOx \(MANTRA\) confirmed and commented:](#)

This is a valid point, and indeed a feature that's missing in the contract. Need to think about it. The reason pools are immutable after creation is to prevent bad actors from creating a pool with favorable conditions/fees to attract liquidity to then change the parameters and manipulate things on their favor, effectively harming the LPs.

Not a high issue, potentially medium.

[3docSec \(judge\) decreased severity to Medium and commented:](#)

I agree Medium makes sense, because pools can leak value with a hypothetical attack path with

stated assumptions, but external requirements like market conditions.

🔗 [M-15] Emergency unlocking penalty makes long duration positions economically advantageous

Submitted by [Lambda](#), also found by [Evo](#)

The farm-manager contract has a static emergency unlock penalty (initialized to 2% in the deployment file `deploy_mantra_dex.sh`) regardless of the position's unlocking duration. However, longer unlocking durations provide significantly higher reward weight multipliers (up to 16x for 1 year lockups). This creates an economic imbalance where users are incentivized to:

1. Create positions with maximum unlocking duration to get the highest weight multiplier (up to 16x).
2. Emergency unlock when they want liquidity, only paying the fixed 2% penalty.

This undermines the intended lockup mechanism since users can get much higher rewards while maintaining effective liquidity through emergency unlocks. The impact is that the protocol's liquidity stability guarantees are weakened, as users are economically incentivized to game the system rather than maintain their intended lock periods.

Moreover, it can be profitable to open a huge position 1 second before an epoch ends and withdraw immediately (in the new epoch), which hurts real users of the system.

🔗 Proof of Concept

The key components of this issue are:

1. Emergency unlock penalty is static, set during initialization [here](#).
2. Weight calculation increases significantly with lock duration [here](#).

For example:

```
#[test]
fn test_calculate_weight() {
  // 1 day lockup = 1x multiplier
  let weight = calculate_weight(&coin(100, "uwhale"), 86400u64).unwrap();
  assert_eq!(weight, Uint128::new(100));

  // 1 year lockup = ~16x multiplier
  let weight = calculate_weight(&coin(100, "uwhale"), 31556926).unwrap();
  assert_eq!(weight, Uint128::new(1599));
}
```

Consider this scenario:

1. User creates position with 1000 tokens and 1 year lock, getting ~16x weight (16000).
2. After collecting boosted rewards, user emergency exits paying only 2% (20 tokens).
3. Net result: User got 16x rewards while maintaining effective liquidity with minimal penalty.

This makes it economically optimal to always choose maximum duration and emergency unlock when needed, defeating the purpose of tiered lockup periods.

🔗 Recommended mitigation steps

The emergency unlock penalty should scale with:

1. Remaining lock duration.

2. Position's weight multiplier.

Suggested formula:

```
emergency_penalty = base_penalty * (remaining_duration / total_duration) * (position_weight / base_weight)
```

This would make emergency unlocks proportionally expensive for positions with higher weights and longer remaining durations, better aligning incentives with the protocol's goals.

Alternative mitigations:

- Cap maximum unlock duration to reduce exploitability.
- Increase base emergency unlock penalty.
- Add minimum hold period before emergency unlocks are allowed.

The key is ensuring the penalty properly counterbalances the increased rewards from longer lock periods.

[jvr0x \(MANTRA\) confirmed and commented:](#)

This is a valid concern and the team is aware of that. However, the way the team intends to instantiate the farm manager at the beginning is to set min and max unlock periods to 1 day, so in that case all positions would be the same.

Will probably address this issue once/if the team decides to increase the constraint.

[M-16] Liquidity providers can lose tokens due to disproportionate deposits not being properly handled

Submitted by [honey-k12](#), also found by [Bauchibred](#), [jasonxiale](#), [Lambda](#), [Lambda](#), [LonnyFlash](#), and [oakcobalt](#)

When providing liquidity to a pool that already has liquidity, users may lose a portion of their deposited tokens if they provide tokens in different proportions relative to the current pool reserves. While the `slippage_tolerance` parameter protects against receiving too few LP tokens, it doesn't protect against token loss due to disproportionate deposits.

The `provide_liquidity` function in the pool manager calculates LP tokens to mint based on the minimum share ratio of provided tokens. When tokens are provided in different proportions relative to the pool's current reserves, the excess tokens from the higher proportion are effectively donated to the pool. This way, users can lose tokens when providing liquidity with disproportionate amounts.

Proof of Concept

First liquidity provider in a pool may provide arbitrary token amounts and set the initial price, but all other liquidity providers must provide liquidity proportionally to current pool reserves.

Here's the relevant code from `commands.rs::provide_liquidity`:

[commands.rs#L213-L231](#)

```
let mut asset_shares = vec![];

for deposit in deposits.iter() {
    let asset_denom = &deposit.denom;
    let pool_asset_index = pool_assets
        .iter()
        .position(|pool_asset| &pool_asset.denom == asset_denom)
        .ok_or(ContractError::AssetMismatch)?;

    asset_shares.push(
```

```

        deposit
            .amount
            .multiply_ratio(total_share, pool_assets[pool_asset_index].amount),
    );
}

std::cmp::min(asset_shares[0], asset_shares[1])
}

```

Since a pool is made of two tokens and liquidity is provided in both tokens, there's a possibility for a discrepancy: token amounts may be provided in different proportions. When this happens, the smaller of the proportions is chosen to calculate the amount of LP tokens minted.

For each deposited token, it calculates a share ratio:

```
share_ratio = deposit_amount * total_share / pool_asset_amount
```

Then it takes the minimum of these share ratios to determine LP tokens to mint:

```
final_share = min(share_ratio_token_a, share_ratio_token_b)
```

As a result, the difference in proportions will create an excess of tokens that won't be redeemable for the amount of LP tokens minted. The excess of tokens gets, basically, donated to the pool: it'll be shared among all liquidity providers of the pool.

While the `slippage_tolerance` argument of the `provide_liquidity` function allows liquidity providers to set the minimal amount of LP tokens they want to receive, it doesn't allow them to minimize the disproportion of token amounts or avoid it at all.

🔗 Recommended mitigation steps

In the `provide_liquidity` function, consider calculating optimal token amounts based on the amounts specified by user, current pool reserves, and the minimal LP tokens amount specified by user. As a reference, consider this piece from the Uniswap V2 Router: [UniswapV2Router02.sol#L45-L60](#).

jvr0x (MANTRA) acknowledged

[3docSec \(judge\) decreased severity to Medium and commented:](#)

I consider this group a valid medium, basing on the following facts:

- As said [here](#), frontrunning is difficult to automate.
- There is slippage protection in place that is sufficient to avoid considerable losses.
- Because extra tokens are not returned; however, the “accidental” frontrun case can lead to non-dust value leakage, which is a solid medium severity impact.

🔗 [M-17] Slippage tolerance vulnerability in StableSwap

Submitted by [DOWSERS](#), also found by [Lambda](#)

The `assert_slippage_tolerance` function does not properly account for the non-linear pricing curve of StableSwap pools, which are influenced by the amplification factor (A). Specifically, the function calculates slippage tolerance based solely on a simple ratio of total deposits and pool tokens without integrating the StableSwap invariant (D). This oversight can lead to the acceptance of transactions with higher actual slippage than allowed, exposing the protocol to user losses or economic inefficiencies.

🔗 Link to code

</contracts/pool-manager/src/helpers.rs#L423>

```

match pool_type {
    PoolType::StableSwap { .. } => {
        let pools_total: Uint256 = pools

```

```

        .into_iter()
        .fold(Uint256::zero(), |acc, x| acc.checked_add(x).unwrap());

let deposits_total: Uint256 = deposits
    .into_iter()
    .fold(Uint256::zero(), |acc, x| acc.checked_add(x).unwrap());

let pool_ratio = Decimal256::from_ratio(pools_total, pool_token_supply);
let deposit_ratio = Decimal256::from_ratio(deposits_total, amount);

if pool_ratio * one_minus_slippage_tolerance > deposit_ratio {
    return Err(ContractError::MaxSlippageAssertion);
}
}
}

```

Impact

Failure to account for the StableSwap invariant (D) and amplification factor (A) allows for:

1. **Incorrect slippage validation:** Transactions with actual slippage exceeding the user-defined tolerance may be accepted.
2. **User losses:** Users may incur unexpected losses due to high slippage.

Example: Incorrect slippage validation:

- Initial pool: [1000, 1000].
- Amplification factor (A): 100.
- User deposit: [100, 50].
- Slippage tolerance: 1% (0.01).
- `pool_token_supply`: 2000.
- Amount (pool token received): 150.

Steps:

1. Calculate total values:

- `pools_total = 1000 + 1000 = 2000`.
- `deposits_total = 100 + 50 = 150`.

2. Calculate ratios:

- `pool_ratio = pools_total / pool_token_supply = 2000 / 2000 = 1.0`.
- `deposit_ratio = deposits_total / amount = 150 / 150 = 1.0`.

3. Slippage check:

- `pool_ratio * (1 - slippage_tolerance) = 1.0 * 0.99 = 0.99`.
- `deposit_ratio = 1.0`.
- Result: `0.99 > 1.0` is false, so the transaction is accepted.

Real slippage using StableSwap invariant

- Before deposit: `Dinitial` = calculation based on [1000,1000] and `A` = 100.

Let's say `Dinitial` = 2000 (perfect equilibrium)

- After deposit: $[1000 + 100, 1000 + 50] = [1100, 1050]$.

D_{final} = StableSwap invariant recalculated

Let's say $D_{final} = 2105$.

Relative change:

- Actual slippage = $(D_{final} - D_{initial}) / D_{initial} = (2105 - 2000) / 2000 = 0.0525$ (5.25%).

Comparison with Tolerance:

- Specified tolerance : 1% (0.01).
- Actual slippage = 5.25% > 1%, so the transaction should have been rejected.

🔗 Risk

- **Likelihood:** Medium, the vulnerability depends on specific deposit patterns, such as highly imbalanced deposits.
- **Impact:** Medium, users face financial losses.

🔗 Recommended mitigation steps

Implement a StableSwap-specific slippage calculation that incorporates the invariant (D) and amplification factor (A):

- Calculate (D) before and after the deposit.
- Derive the price impact based on (D) and compare it to the user-defined slippage tolerance.

jvr0x (MANTRA) confirmed

🔗 [M-18] Stablepools return wrong price when they do not converge

Submitted by [DadeKuma](#)

[/contracts/pool-manager/src/helpers.rs#L751](#)

🔗 Finding description and impact

A StableSwap pool calculates the price using Newton's method to approximate the D value. This calculation might not converge in unbalanced pools, resulting in a wrong price, and in this case, it shouldn't be possible to swap.

However, this scenario is not prevented as the transaction will not revert even when the function does not converge.

🔗 Proof of Concept

The computation of D converges when the result is either 1 or 0; otherwise, it diverges. In the latter case, the pool is considered nonfunctional, as the StableSwap math will not work as intended.

In case a pool diverges, it shouldn't be possible to swap, but only to withdraw liquidity. This is implemented correctly in the original Curve [implementation](#).

But in Mantra this is not the case, as the result is returned even if the computation did not converge:

```
pub fn compute_d(amp_factor: &u64, deposits: &[Coin]) -> Option<Uint512> {
    let n_coins = Uint128::from(deposits.len() as u128);
```

```

        // sum(x_i), a.k.a S
        let sum_x = deposits
            .iter()
            .fold(Uint128::zero(), |acc, x| acc.checked_add(x.amount).unwrap());

        if sum_x == Uint128::zero() {
            Some(Uint512::zero())
        } else {
            // do as below but for a generic number of assets
            let amount_times_coins: Vec<Uint128> = deposits
                .iter()
                .map(|coin| coin.amount.checked_mul(n_coins).unwrap())
                .collect();

            // Newton's method to approximate D
            let mut d_prev: Uint512;
            let mut d: Uint512 = sum_x.into();
            for _ in 0..256 {
                let mut d_prod = d;
                for amount in amount_times_coins.clone().into_iter() {
                    d_prod = d_prod
                        .checked_mul(d)
                        .unwrap()
                        .checked_div(amount.into())
                        .unwrap();
                }
                d_prev = d;
                d = compute_next_d(amp_factor, d, d_prod, sum_x, n_coins).unwrap();
                // Equality with the precision of 1
                if d > d_prev {
                    if d.checked_sub(d_prev).unwrap() <= Uint512::one() {
                        break;
                    }
                } else if d_prev.checked_sub(d).unwrap() <= Uint512::one() {
                    break;
                }
            }

            -> Some(d)
        }
    }
}

```

</contracts/pool-manager/src/helpers.rs#L751>

🔗 Recommended mitigation steps

Consider the following fix:

```

        d = compute_next_d(amp_factor, d, d_prod, sum_x, n_coins).unwrap();
        // Equality with the precision of 1
        if d > d_prev {
            if d.checked_sub(d_prev).unwrap() <= Uint512::one() {
                - break;
                + return Some(d);
            }
        } else if d_prev.checked_sub(d).unwrap() <= Uint512::one() {
            - break;
            + return Some(d);
        }
    }

    - Some(d)
    + Err(ContractError::ConvergeError)

```

[Abdessamed \(warden\) commented:](#)

In case a pool diverges, it shouldn't be possible to swap, but only to withdraw liquidity.

This is incorrect, due to the StableSwap math, there is no straightforward algebraic formula to compute D and instead, Newton's method is used to estimate the value. In case the pool is highly imbalanced and Newton's method does not converge, the latest estimated value should be returned

rather than reverting the transaction. The pool can easily be balanced again by adding balanced liquidity in subsequent transactions.

The StableSwap pool referenced by the warden is `Zaps` pool and it is a non-standard pool. You can see that in `3pool` which is one of the most used pools in Curve, the `get_D` does NOT revert and instead returns the best-estimated value

[DadeKuma \(warden\) commented:](#)

If Newton's method does not converge, it will return the wrong price; this is simply how an iterative algorithm works. You shared a specific pool implementation: a 3-token pool designed for DAI/USDC/USDT, as seen in [this comment](#).

What [I shared](#) is a normal 2-token stable pool that requires this check, which is definitely standard. For example [EURS/sEUR](#) or [the pool template](#) from the same repository you have shared.

[jvrOx \(MANTRA\) confirmed and commented:](#)

DadeKuma - does it mean there has to be two cases for when the pool has 2 vs 3 assets?

[DadeKuma \(warden\) commented:](#)

@jvrOx - it's not related to the number of assets but rather to the expected stability of the pool. For example, with DAI/USDC/USDT, it's basically impossible for the algorithm to diverge, making this check unnecessary (unlike UST/DAI/USDC/USDT which also has [this check](#)).

However, since this function can be used with any asset in 2, 3, or 4-asset combinations, I recommend implementing this check in any case.

[jvrOx \(MANTRA\) commented:](#)

Seems reasonable.

[\[M-19\] Vulnerable liquidity slippage calculation doesn't ensure slippage protection due to unscaled assets sum](#)

Submitted by [oakcobalt](#)

[/contracts/pool-manager/src/helpers.rs#L426](#)

[/contracts/pool-manager/src/helpers.rs#L429](#)

[Finding description and impact](#)

Current slippage calculation for providing liquidity to stableswap pool uses simple sum of unscaled asset balance. This is vulnerable because it doesn't account for differences in asset decimals, which might cause slippage calculation to be invalid.

[Proof of Concept](#)

Vulnerable case: `(10000000 usdc + 1e18 dai)` vs `(1e18 usdc + 1e6 dai)`

The pool impact of depositing `1000000 usdc + 1e18 dai` vs depositing `1e18 usdc + 1e6 dai` are expected to diverge.

However, in the current slippage check, both cases will lead to the same `deposits_total`. This invalidates the `pool_ratio / deposit_ratio` calculation.

```
//contracts/pool-manager/src/helpers.rs

pub fn assert_slippage_tolerance(
    slippage_tolerance: &Option<Decimal>,
    deposits: &[Coin],
    pools: &[Coin],
    pool_type: PoolType,
    amount: Uint128,
    pool_token_supply: Uint128,
) -> Result<(), ContractError> {
    if let Some(slippage_tolerance) = *slippage_tolerance {
        ...
        let deposits: Vec<Uint256> = deposits.iter().map(|coin| coin.amount.into()).collect();
        let pools: Vec<Uint256> = pools.iter().map(|coin| coin.amount.into()).collect();
        // Ensure each prices are not dropped as much as slippage tolerance rate
        match pool_type {
            PoolType::StableSwap { .. } => {
                let pools_total: Uint256 = pools
                    .into_iter()
                    .fold(Uint256::zero(), |acc, x| acc.checked_add(x).unwrap());
                let deposits_total: Uint256 = deposits
                    .into_iter()
                    .fold(Uint256::zero(), |acc, x| acc.checked_add(x).unwrap());

                let pool_ratio = Decimal256::from_ratio(pools_total, pool_token_supply);
                let deposit_ratio = Decimal256::from_ratio(deposits_total, amount);

                // the slippage tolerance for the stableswap can't use a simple ratio for calculating
                // slippage when adding liquidity. Due to the math behind the stableswap, the amp factor
                // needs to be in as well, much like when swaps are done
                if pool_ratio * one_minus_slippage_tolerance > deposit_ratio {
                    return Err(ContractError::MaxSlippageAssertion);
                }
            }
        }
    }
}
```

[/contracts/pool-manager/src/helpers.rs#L424-L429](#)

Flows: `contracts/pool-manager/src/liquidity/commands::provide_liquidity` → `helpers::assert_slippage_tolerance`

As we can see, in the vulnerable case above, `pools_total` and `deposits_total` will be the same in both cases where the actual deposits assets amount when aligned to 18 decimals and the actual deposit assets value vastly differ. This invalidates the comparison check results.

🔗 Recommended mitigation steps

Scale the asset to the same decimal before performing sum.

jvr0x (MANTRA) confirmed via chat with C4 staff

🔗 Low Risk and Non-Critical Issues

For this audit, 11 reports were submitted by wardens detailing low risk and non-critical issues. The [report highlighted below](#) by Bauchibred received the top score from the judge.

The following wardens also submitted reports: [Oxcb90f054](#), [axelot](#), [Daniel526](#), [jasonxiale](#), [Lambda.oakcobalt](#), [OxElliot](#), [Rhaydden](#), [Sparrow](#), and [Tigerfrake](#).

🔗 Table of Contents

Issue ID	Description
[01]	Unvalidated genesis epoch update in Epoch Manager allows manipulation of farm rewards
[02]	Reduced Newton-Raphson iterations easily leads to slightly incorrect results due to potential precision

Issue ID	Description
	loss
[03]	Inflation attack protection is not really sufficient
[04]	Refunds of fees being paid should be processed for all assets
[05]	Validation of the token factory fee should accept cumulative payments
[06]	Having a hardcoded maximum slippage should be rethought
[07]	Wrong counter load while creating positions unnecessarily hikes cost for users creating positions with explicit identifiers
[08]	Stableswap pools are allowed to have an amp value of 0 which would cause a DOS to swaps on pools
[09]	Consider allowing a change in the unlocking duration if within valid range when expanding a position
[10]	Consider upgrading to CosmWasm 2.2.0 for enhanced migration capabilities
[11]	Allow for the reduction of the <code>max_concurrent_farms</code> when updating the config
[12]	Consider enforcing first liquidity provider to be pool creator
[13]	Wrong pool asset length should be correctly flagged during slippage tolerance assertion
[14]	Approach of position creation should fail faster when receiver has exceeded their position limit
[15]	Fix incorrect documentation for position creation authorization check
[16]	Emergency unlock penalty should not be allowed to be up to 100%
[17]	Inaccurate year calculation has been set in as AMM constants
[18]	Inconsistent farm management permissions in sister operations
[19]	Consider conjugating the pool asset lengths validation when creating a pool
[20]	Fix typos
[21]	Remove redundant checks when withdrawing a position
[22]	Wrong code/documentation about fee
[23]	Use <code>assert_admin</code> or remove it since we have the <code>cw-ownable</code> being used
[24]	Protocol should be deployment ready

🔗 [01] Unvalidated genesis epoch update in Epoch Manager allows manipulation of farm rewards

The `update_config` function in the epoch manager contract allows updating the entire epoch configuration without proper validation of the genesis epoch:

</contracts/epoch-manager/src/commands.rs#L19-L23>

```
if let Some(epoch_config) = epoch_config.clone() {
    validate_epoch_duration(epoch_config.duration)?;
    config.epoch_config = epoch_config; // Entire config replaced without genesis validation
    CONFIG.save(deps.storage, &config);
}
```

While the duration is validated, there are no checks on the new genesis epoch value. This contrasts with the instantiation where genesis epoch is properly validated:

```
ensure!(
    msg.epoch_config.genesis_epoch.u64() >= env.block.time.seconds(),
    ContractError::InvalidStartTime
);
```

🔗 Impact

Protocol Invariant Violation: From the protocol [documentation](#):

```
### Main invariants
```

Main invariants:

- Genesis epoch and epoch duration, not intended to be changed once set up.

The current implementation violates this core invariant.

Also, since farm rewards are calculated based on epochs, changing the genesis epoch can manipulate reward distributions:

- Setting genesis epoch to the future could temporarily freeze reward distributions.
- Setting it to the past could cause incorrect reward calculations and potentially drain farm rewards.

Changing genesis epoch after farms are active disrupts the continuous epoch sequence.

🔗 Recommended Mitigation Steps

Add proper validation:

- Ensure new genesis epoch maintains continuity with existing epochs.
- Validate impact on active farm positions.
- Add checks for reward calculation consistency.

🔗 [02] Reduced Newton-Raphson iterations easily leads to slightly incorrect results due to potential precision loss

The Mantra DEX implementation has reduced the number of Newton-Raphson iterations from Curve's original 256 to 32:

</contracts/pool-manager/src/helpers.rs#L15-17>

```
/// The amount of iterations to perform when calculating the Newton-Raphson approximation.
const NEWTON_ITERATIONS: u64 = 32;
```

This reduction affects two critical calculations:

1. Computing the invariant D in `compute_d`:

```
// Newton's method to approximate D
let mut d_prev: Uint512;
let mut d: Uint512 = sum_x.into();
for _ in 0..32 { // Original Curve uses 256
    let mut d_prod = d;
    for amount in amount_times_coins.clone().into_iter() {
        d_prod = d_prod
            .checked_mul(d)
            .unwrap()
            .checked_div(amount.into())
            .unwrap();
    }
    d_prev = d;
    d = compute_next_d(amp_factor, d, d_prod, sum_x, n_coins).unwrap();
    // Equality with the precision of 1
    if d > d_prev {
        if d.checked_sub(d_prev).unwrap() <= Uint512::one() {
            break;
        }
    } else if d_prev.checked_sub(d).unwrap() <= Uint512::one() {
        break;
    }
}
```

2. Computing swap amounts in `compute_y_raw`:

```

let mut y_prev: Uint512;
let mut y = d;
for _ in 0..32 { // Original uses more iterations
    y_prev = y;
    let y_numerator = y.checked_mul(y).unwrap().checked_add(c).unwrap();
    let y_denominator = y
        .checked_mul(Uint512::from(2u8))
        .unwrap()
        .checked_add(b)
        .unwrap()
        .checked_sub(d)
        .unwrap();
    y = y_numerator.checked_div(y_denominator).unwrap();
    // Check convergence
    if |y - y_prev| <= 1 break;
}

```

While Newton-Raphson typically converges quadratically (error is squared each iteration), this approach assumes the initial guess is sufficiently close to the root

However, in DeFi pools:

1. Extreme pool imbalances can occur.
2. Large trades can push the system far from equilibrium.
3. Precision is critical for fair pricing and arbitrage.

Would be key to note that the original 256 iterations on Curve weren't really arbitrary and were also set up for stablecoin pools:

[/contracts/pool-templates/base/SwapTemplateBase.vy#L445-L508](#)

```

for _i in range(255):
    D_P: uint256 = D
    for _x in _xp:
        D_P = D_P * D / (_x * N_COINS) # If division by 0, this will be borked: only withdrawal will v
    Dprev = D
    D = (Ann * S / A_PRECISION + D_P * N_COINS) * D / ((Ann - A_PRECISION) * D / A_PRECISION + (N_COINS
    # Equality with the precision of 1
    if D > Dprev:
        if D - Dprev <= 1:
            return D
    else:
        if Dprev - D <= 1:
            return D
    # convergence typically occurs in 4 rounds or less, this should be unreachable!
    # if it does happen the pool is borked and LPs can withdraw via `remove_liquidity`
    raise

```

This is because they ensured convergence even in extreme edge cases where:

- Pool ratios are highly skewed.
- Large trades significantly impact pool balance.
- Initial guesses are far from the solution.
- Multiple solutions exist and we need the correct one.

🔗 Impact

QA, considering this can be argued as intended implementation; however, subtle issues could be incorrect price calculations and as such unfair trades and even failed arbitrage opportunities, as this is all cumulative.

🔗 Recommended Mitigation Steps

Consider increasing the number of iterations in the Newton-Raphson approximation.

🔗 [03] Inflation attack protection is not really sufficient

The Mantra DEX attempts to protect against first depositor attacks by implementing a minimum liquidity mechanism, similar to that of Uniswap.

Before going further we can have an overview of the bug case here:

- <https://blog.openzeppelin.com/a-novel-defense-against-erc4626-inflation-attacks>
- <https://mixbytes.io/blog/overview-of-the-inflation-attack>

And we know that in order for this attack to be possible, two conditions need to be met:

1. Malicious user mint 1 mint of share.
2. Donate or transfer assets to the vault to inflate the assets per share.

However, the implementation has a flaw in how these minimum liquidity tokens that are expected to be used to protect against the 1 mint of share are stored/sent:

</contracts/pool-manager/src/liquidity/commands.rs#L184-L213>

```
if total_share == Uint128::zero() {  
    // Make sure at least MINIMUM_LIQUIDITY_AMOUNT is deposited to mitigate the risk of the  
    // depositor preventing small liquidity providers from joining the pool  
    let share = Uint128::new(  
        (U256::from(deposits[0].amount.u128())  
        .checked_mul(U256::from(deposits[1].amount.u128()))  
        .ok_or::<ContractError>(  
            ContractError::LiquidityShareComputationFailed,  
        ))?  
        .integer_sqrt()  
        .as_u128(),  
    )  
    .saturating_sub(MINIMUM_LIQUIDITY_AMOUNT);  
  
    // share should be above zero after subtracting the MINIMUM_LIQUIDITY_AMOUNT  
    if share.is_zero() {  
        return Err(ContractError::InvalidInitialLiquidityAmount(  
            MINIMUM_LIQUIDITY_AMOUNT,  
        ));  
    }  
  
    messages.push(amm::lp_common::mint_lp_token_msg(  
        liquidity_token.clone(),  
        &env.contract.address,  
        &env.contract.address,  
        MINIMUM_LIQUIDITY_AMOUNT,  
    ));  
  
    share
```

That's to say in the case the case we have zero shares then we send the minimum liquidity amount to the contract instead of to the burn/dead address as is done in Uniswap.

This creates a vulnerability that could be exploited through the following steps:

Below we subtly assume that admin introduces a new functionality that could then allow for access to this contract owned tokens.

1. Contract is migrated to a new version that allows owner to withdraw contract-owned LP tokens and allows for updating pool asset balance from direct transfers.
2. Owner withdraws the `MINIMUM_LIQUIDITY_AMOUNT` tokens from the contract.
3. Assume pool is not really active and we now have one user left.
4. User is malicious so they can withdraw all but one of their liquidity tokens.
5. Since `total_share` is not exactly zero but effectively zero, new deposits skip this check:

```

if total_share == Uint128::zero() {
    // Minimum liquidity check only happens on first deposit
    // But total_share is not zero, just very small
    ...
}

```

6. First depositor attack becomes possible again since minimum liquidity protection is bypassed.

Impact

Impact is quite high as we then are back open to first depositor attack, since after the user withdraws back and leaves just 1 share, the next depositor would receive a massively unfair exchange rate which allows the malicious user to backrun the tx and skew off these assets. Albeit, we have to classify as QA, considering currently there is no how we can access directly the minimum liquidity tokens, i.e., funds in the contract and it needs to be via an introduced function in an upgrade or some sort.

Would be key to note that besides the admin window we could still have an issue where there is a mismatch between the amount of pool assets and the amount of minted lp tokens; which is because when minting the lp tokens, we use the `checked_add` function, which does not overflow, so the last user that deposits and every that deposits to pass the maximum value would then cause a mismatch in the amount of pool assets and the amount of lp tokens minted:

</contracts/pool-manager/src/liquidity/commands.rs#L376-L390>

```

// Increment the pool asset amount by the amount sent
for asset in deposits.iter() {
    let asset_denom = &asset.denom;
    let pool_asset_index = pool_assets
        .iter()
        .position(|pool_asset| &pool_asset.denom == asset_denom)
        .ok_or(ContractError::AssetMismatch)?;

    pool_assets[pool_asset_index].amount = pool_assets[pool_asset_index]
        .amount
        .checked_add(asset.amount)?;
}

pool.assets = pool_assets.clone();

```

Recommended Mitigation Steps

Follow Uniswap's approach of permanently burning minimum liquidity tokens, this can be done by using the same approach as to when liquidity is being withdrawn:

```

// Burn the LP tokens
messages.push(amm::lp_common::burn_lp_asset_msg(
    liquidity_token,
    env.contract.address,
    amount,
)?);

```

[04] Refunds of fees being paid should be processed for all assets

</contracts/farm-manager/src/helpers.rs#L17-L89>

```

pub(crate) fn process_farm_creation_fee(
    config: &Config,
    info: &MessageInfo,
    params: &FarmParams,
) -> Result<Vec<CosmosMsg>, ContractError> {
    // ..snip

    match paid_fee_amount.cmp(&farm_creation_fee.amount) {
        // ..snip
        Ordering::Greater => {
            // if the user is paying more than the farm_creation_fee, check if it's trying to create

```

```

        // a farm with the same asset as the farm_creation_fee.
        // otherwise, refund the difference
        if farm_creation_fee.denom == params.farm_asset.denom {
            // check if the amounts add up, i.e. the fee + farm asset = paid amount. That is because the
            // and the creation fee asset are the same, all go in the info.funds of the transaction

            ensure!(
                params
                    .farm_asset
                    .amount
                    .checked_add(farm_creation_fee.amount)?
                    == paid_fee_amount,
                ContractError::AssetMismatch
            );
        } else {
            let refund_amount = paid_fee_amount.saturating_sub(farm_creation_fee.amount);

            messages.push(
                BankMsg::Send {
                    to_address: info.sender.clone().into_string(),
                    amount: vec![Coin {
                        amount: refund_amount,
                        denom: farm_creation_fee.denom.clone(),
                    }],
                }
                .into(),
            );
        }
    }
}

// send farm creation fee to fee collector
if farm_creation_fee.amount > Uint128::zero() {
    messages.push(
        BankMsg::Send {
            to_address: config.fee_collector_addr.to_string(),
            amount: vec![farm_creation_fee.to_owned()],
        }
        .into(),
    );
}

Ok(messages)
}

```

This function is used in order to process the farm creation fee. Now it includes the logic to see if the fee being paid is the same denom as the market that's been created; however, the issue is that whereas we can process the refund when the fee is different from the asset being used to create the market, we can't do this when they are the same and instead have a strict requirement that they must be equal.

🔗 Recommended Mitigation Steps

In both cases ensure at least the necessary amount is provided and if otherwise then refund the difference.

🔗 [05] Validation of the token factory fee should accept cumulative payments

The pool manager's fee validation logic incorrectly requires the full token factory fee to be paid *in a single accepted denomination*, even when the user provides the equivalent total amount split across multiple accepted denoms.

In order to validate that fees are paid correctly for the transactions, we use `validate_fees_are_paid()`.

Now in the case where the pool fee denom is one of the token factory fee denoms BUT there are multiple token factory fee options, we get into this window:

</contracts/pool-manager/src/helpers.rs#L575-L597>

```

// Check if the user paid the token factory fee in any other of the allowed denoms
let tf_fee_paid = denom_creation_fee.iter().any(|fee| {
    let paid_fee_amount = info
        .funds
        .iter()
        .filter(|fund| fund.denom == fee.denom)

```

```

        .map(|fund| fund.amount)
        .try_fold(Uint128::zero(), |acc, amount| acc.checked_add(amount))
        .unwrap_or(Uint128::zero());

    total_fees.push(Coin {
        denom: fee.denom.clone(),
        amount: paid_fee_amount,
    });

    paid_fee_amount == fee.amount
});

ensure!(tf_fee_paid, ContractError::TokenFactoryFeeNotPaid);

```

The issue is that the code uses `.any()` to check if ANY SINGLE denomination matches the full required fee amount. This means that even if a user provides the equivalent value split across multiple accepted denoms, the validation will fail.

For example, if the required fee is 100 tokens and there are 5 accepted denoms:

- User pays 20 tokens in each of the 5 accepted denoms (total value = 100).
- For each denom check: `paid_fee_amount (20) == fee.amount (100)` is `false`.
- Since no single denom has the full amount, `any()` returns `false`.
- Transaction reverts with `TokenFactoryFeeNotPaid` error.

This happens even though the user has provided the full required fee value, just split across different denoms, which is counterintuitive to the intended [implementation](#).

🔗 Impact

Broken implementation of the validation logic, considering that users are forced to pay the entire fee in a single denomination even when they have sufficient funds split across multiple accepted denoms

🔗 Recommended Mitigation Steps

Modify the validation to accept cumulative payments across accepted denoms.

🔗 [06] Having a hardcoded maximum slippage should be rethought

The Mantra DEX implements a hardcoded maximum slippage cap that can prevent legitimate trades and force users into unfavorable positions, especially during high volatility or for certain asset pairs.

The issue exists in the swap [implementation](#):

```

/// Cap on the maximum swap slippage that is allowed. If max_spread goes over this limit, it will
/// be capped to this value.
pub const MAX_ALLOWED_SLIPPAGE: &str = "0.5";

```

This value is enforced in the `assert_max_spread` function:

```

let max_spread: Decimal256 = max_spread
    .unwrap_or(Decimal::from_str(DEFAULT_SLIPPAGE)?)
    .min(Decimal::from_str(MAX_ALLOWED_SLIPPAGE)?)
    .into();

```

This however creates several problems:

1. **Forced Cap During Volatility:** Even if a user explicitly accepts higher slippage, their `max_spread` is capped at 50%:

```

// User input: max_spread = 0.75 (75% slippage acceptance)
// After cap: max_spread = 0.50 (50% slippage cap)

```

2. Single Cap For All Assets: The same 50% cap applies to all asset pairs, regardless of their:

- Historical volatility
- Liquidity depth
- Market conditions
- Trading volume

🔗 Impact

Users cannot execute valid trades during high volatility which directly translates to loss of funds 💰, considering if the market is on a free fall and they are attempting to sell off their position, it always reverts during the assertion of the maximum slippage cap, leaving them to hold on to their assets for longer and more loss of funds.

🔗 Recommended Mitigation Steps

Remove the hard cap and instead allow a dynamic slippage.

🔗 [07] Wrong counter load while creating positions unnecessarily hikes cost for users creating positions with explicit identifiers

Take a look at the position creation logic in `position/commands.rs`:

</contracts/farm-manager/src/position/commands.rs#L57-L62>

```
pub(crate) fn create_position() {  
  
    //snip  
  
    // Counter is loaded before checking if we need it  
    let position_id_counter = POSITION_ID_COUNTER  
        .may_load(deps.storage)?  
        .unwrap_or_default()  
        + 1u64;  
  
    // compute the identifier for this position  
    let identifier = if let Some(identifier) = identifier {  
        // For explicit IDs, counter is never used  
        format!("{EXPLICIT_POSITION_ID_PREFIX}{identifier}")  
    } else {  
        // Only auto-generated IDs use the counter  
        POSITION_ID_COUNTER.save(deps.storage, &position_id_counter)?;  
        format!("{AUTO_POSITION_ID_PREFIX}{position_id_counter}")  
    };  
}
```

The code unnecessarily loads and increments the position counter before checking if it's needed. When creating a position with an explicit identifier, this counter value is never used.

🔗 Impact

Every position creation with an explicit identifier performs an unnecessary:

1. Storage read operation (`may_load`).
2. Arithmetic operation (`+ 1u64`).
3. Memory allocation for the counter.

🔗 Recommended Mitigation Steps

Move the counter loading inside the `else` branch where it's actually needed:

```
let identifier = if let Some(identifier) = identifier {  
    // For explicit IDs, just format with prefix  
    format!("{EXPLICIT_POSITION_ID_PREFIX}{identifier}")  
}
```

```

    } else {
      // Only load and increment counter for auto-generated IDs
      let position_id_counter = POSITION_ID_COUNTER
        .may_load(deps.storage)?
        .unwrap_or_default()
        + 1u64;
      POSITION_ID_COUNTER.save(deps.storage, &position_id_counter)?;
      format!("{AUTO_POSITION_ID_PREFIX}{position_id_counter}")
    };
  };

```

This change:

1. Eliminates unnecessary storage reads for explicit identifiers.
2. Makes the code more efficient.
3. Maintains the same functionality.

🔗 [O8] Stableswap pools are allowed to have an amp value of 0 which would cause a DOS to swaps on pools

</contracts/pool-manager/schema/pool-manager.json#L588-L612>

```

"PoolType": {
  "description": "Possible pool types, it can be either a constant product (xyk) pool or a stable swap",
  "oneOf": [
    {
      "description": "A stable swap pool.",
      "type": "object",
      "required": [
        "stable_swap"
      ],
      "properties": {
        "stable_swap": {
          "type": "object",
          "required": [
            "amp"
          ],
          "properties": {
            "amp": {
              "description": "The amount of amplification to perform on the constant product part of",
              "type": "integer",
              "format": "uint64",
              "minimum": 0.0
            }
          }
        },
        "additionalProperties": false
      }
    }
  ]
}

```

We can see that during creation of a stable swap pool the amp value is allowed to be 0, which is not a valid value.

This has been flagged in the forked WhiteWhale audit report previously, see [report here](#). However, the issue is that when we have an amp value of 0, all swaps would fail.

🔗 Impact

Borderline here, considering with an amp value of 0, all swaps would fail causing a DOS in using the pool.

After a further review, this case completely bricks the pool instance and as such a more detailed report has been submitted.

🔗 Recommended Mitigation Steps

Change the min amp value to 1, as done in the [test module](#):

```

/// Minimum amplification coefficient.

```

```
pub const MIN_AMP: u64 = 1;
```

🔗 [09] Consider allowing a change in the unlocking duration if within valid range when expanding a position

</contracts/farm-manager/src/position/commands.rs#L111-L171>

```
pub(crate) fn expand_position(
    deps: DepsMut,
    env: &Env,
    info: MessageInfo,
    identifier: String,
) -> Result<Response, ContractError> {
    let mut position = get_position(deps.storage, Some(identifier.clone()))?.ok_or(
        ContractError::NoPositionFound {
            identifier: identifier.clone(),
        },
    )?;

    let lp_asset = cw_utils::one_coin(&info)?;

    // ensure the lp denom is valid and was created by the pool manager
    let config = CONFIG.load(deps.storage)?;
    validate_lp_denom(&lp_asset.denom, config.pool_manager_addr.as_str())?;

    // make sure the lp asset sent matches the lp asset of the position
    ensure!(
        position.lp_asset.denom == lp_asset.denom,
        ContractError::AssetMismatch
    );

    ensure!(
        position.open,
        ContractError::PositionAlreadyClosed {
            identifier: position.identifier.clone(),
        }
    );

    // ensure only the receiver itself or the pool manager can refill the position
    ensure!(
        position.receiver == info.sender || info.sender == config.pool_manager_addr,
        ContractError::Unauthorized
    );

    position.lp_asset.amount = position.lp_asset.amount.checked_add(lp_asset.amount)?;
    POSITIONS.save(deps.storage, &position.identifier, &position)?;

    // Update weights for the LP and the user
    update_weights(
        deps,
        env,
        &position.receiver,
        &lp_asset,
        position.unlocking_duration,
        true,
    )?;

    Ok(Response::default().add_attributes(vec![
        ("action", "expand_position".to_string()),
        ("receiver", position.receiver.to_string()),
        ("lp_asset", lp_asset.to_string()),
        (
            "unlocking_duration",
            position.unlocking_duration.to_string(),
        ),
    ]))
}
```

The current implementation of the `expand_position` function allows users to add more assets to an existing position. However, it doesn't provide the flexibility to modify the unlocking duration during this process.

🔗 Impact

A user with existing positions wants to increase their investment in a particular asset. Due to constraints (e.g.,

reaching position limits), they must expand an existing position. If their initial unlocking duration was long (e.g., 1 year), they cannot reduce it (e.g., to 3 months) through expansion.

🔗 Recommended Next Steps

Consider investigating the feasibility of incorporating unlocking duration modification during position expansion. This could involve:

- A separate function specifically for modifying unlocking duration within protocol limits.
- Allowing users to specify a new unlocking duration as part of the `expand_position` function.

🔗 [10] Consider upgrading to CosmWasm 2.2.0 for enhanced migration capabilities

Take a look at the current migration implementation in the epoch manager contract:

</contracts/epoch-manager/src/contract.rs#L78-L85>

```
#[entry_point]
pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) -> Result<Response, ContractError> {
    validate_contract!(deps, CONTRACT_NAME, CONTRACT_VERSION);
    set_contract_version(deps.storage, CONTRACT_NAME, CONTRACT_VERSION)?;
    Ok(Response::default())
}
```

The contract currently uses the legacy migration signature from older CosmWasm versions. CosmWasm 2.2.0 introduces a new migration signature that provides additional migration information through the `MigrateInfo` struct:

```
#[cfg_attr(not(feature = "library"), entry_point)]
#[migrate_version(MIGRATE_VERSION)]
pub fn migrate(
    deps: DepsMut,
    env: Env,
    msg: MigrateMsg,
    migrate_info: MigrateInfo
) -> StdResult<Response>
```

🔗 Impact

QA, albeit the current implementation misses out on important migration features that could improve contract maintainability and safety:

NB: Issue is quite rampant across scope as it's the same logic for all migration instances in protocol, this can be confirmed by this [search command](#).

🔗 Recommended Mitigation Steps

Upgrade the protocol's CosmWasm dependency to version 2.2.0 or newer and implement the new migration signature.

🔗 References

- [CosmWasm Documentation - Migrate](#)
- [CosmWasm 2.2.0 Release Notes](#)

🔗 [11] Allow for the reduction of the `max_concurrent_farms` when updating the config

The contract enforces a one-way constraint on the `max_concurrent_farms` parameter in the config update logic:

[/contracts/farm-manager/src/manager/commands.rs#L325-L333](#)

```
if let Some(max_concurrent_farms) = max_concurrent_farms {
    ensure!(
        max_concurrent_farms >= config.max_concurrent_farms,
        ContractError::MaximumConcurrentFarmsDecreased
    );

    config.max_concurrent_farms = max_concurrent_farms;
}
```

This code explicitly prevents any reduction in the `max_concurrent_farms` value, only allowing it to increase or remain unchanged. This creates a permanent state where:

1. Once set to a high value, it cannot be reduced even if market conditions change.
2. The parameter becomes effectively immutable for any downward adjustments.
3. Initial misconfiguration cannot be corrected.

🔗 Impact

- If `max_concurrent_farms` is set high during a period of high market activity, it cannot be reduced when activity decreases.
- This prevents optimal resource allocation and contract tuning based on actual usage patterns.
- Functions that iterate over farms (like `get_farms_by_lp_denom`) must maintain capacity for the maximum value.

🔗 Recommended Mitigation Steps

Remove the non-reduction constraint and replace with proper validation.

🔗 [12] Consider enforcing first liquidity provider to be pool creator

In the `provide_liquidity` function in `contracts/pool-manager/src/liquidity/commands.rs`, any user can be the first liquidity provider to a pool. This creates a potential risk where regular users who are not associated with the project could unknowingly become the first liquidity provider and be subject to minimum liquidity requirements or other initialization parameters that may not be optimal for them.

The first liquidity provider plays a crucial role in:

1. Setting the initial price ratio for the pool.
2. Determining the initial liquidity depth.
3. Establishing baseline parameters for future liquidity providers.

Currently, there is no mechanism to ensure that the pool creator, who understands the pool's design and intended parameters, is the first liquidity provider, also this minimum liquidity requirement is enforced for the initial provider as a method to curb the famous `first depositor attack`; however, this then unfairly costs a user their tokens.

[/contracts/pool-manager/src/liquidity/commands.rs#L182-L213](#)

```
let share = match &pool.pool_type {
    PoolType::ConstantProduct => {
        if total_share == Uint128::zero() {
            // Make sure at least MINIMUM_LIQUIDITY_AMOUNT is deposited to mitigate the risk of the
            // depositor preventing small liquidity providers from joining the pool
            let share = Uint128::new(
                U256::from(deposits[0].amount.u128())
            )
        }
    }
}
```

```

        .checked_mul(U256::from(deposits[1].amount.u128()))
        .ok_or::<ContractError>(
            ContractError::LiquidityShareComputationFailed,
        ))?
        .integer_sqrt()
        .as_u128(),
    )
    .saturating_sub(MINIMUM_LIQUIDITY_AMOUNT);

    // share should be above zero after subtracting the MINIMUM_LIQUIDITY_AMOUNT
    if share.is_zero() {
        return Err(ContractError::InvalidInitialLiquidityAmount(
            MINIMUM_LIQUIDITY_AMOUNT,
        ));
    }

    messages.push(amm::lp_common::mint_lp_token_msg(
        liquidity_token.clone(),
        &env.contract.address,
        &env.contract.address,
        MINIMUM_LIQUIDITY_AMOUNT,
    ));

    share
} else {

```

Impact

QA, considering this is a common pattern in DeFi protocols to help curb the first depositor attack window; however, enforcing the creator is the first liquidity provider easily sorts a lot of stuff as we can ensure they also set the correct price for the pool.

Recommendations

Implement a mechanism to ensure the pool creator is the first liquidity provider.

[13] Wrong pool asset length should be correctly flagged during slippage tolerance assertion

In `helpers.rs`, when asserting the slippage tolerance, the error message for invalid pool asset length only shows the deposit length, even when the pool length is the actual problem:

</contracts/pool-manager/src/helpers.rs#L441-L447>

```

if deposits.len() != 2 || pools.len() != 2 {
    return Err(ContractError::InvalidPoolAssetsLength {
        expected: 2,
        actual: deposits.len(), // Only shows deposits.len() even if pools.len() is wrong
    });
}

```

For example, if:

- `deposits.len() == 2` (correct)
- `pools.len() == 3` (incorrect)

The error will still say “expected: 2, actual: 2”, which is misleading since the actual problem is with the pool length being 3.

Impact

This can lead to confusion during debugging as developers will be looking at the deposit length when the actual issue might be with the pool length.

Recommended Mitigation Steps

Modify the error to clearly indicate which length is incorrect:.

🔗 [14] Approach of position creation should fail faster when receiver has exceeded their position limit

Take a look at the position creation function in `position/commands.rs`:

</contracts/farm-manager/src/position/commands.rs#L25-L109>

► Details

Evidently, the position limit validation check is placed near the end of the function:

```
// Validate position limit only after all other operations
validate_positions_limit(deps.as_ref(), &receiver, true)?;
```

This means that if a user has exceeded their maximum allowed positions:

1. The function will execute all prior operations (validations, calculations, state reads).
2. Only then check if the user has exceeded their position limit.
3. Finally revert the transaction.

🔗 Impact

QA, whereas the implementation is not *broken*, the current placement of the validation check causes unnecessary computation and state reads when a user has exceeded their position limit. Moving this check to the start of the function would make the implementation more efficient by failing fast.

🔗 Recommended Mitigation Steps

Move the position limit validation to the start of the function, right after receiver validation:

pseudo implementation:

```
pub fn create_position(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    receiver: Option<String>,
    lp_token: String,
    unlocking_duration: u64,
) -> Result<Response, ContractError> {
    let config = CONFIG.load(deps.storage)?;

    // Validate and get receiver early
    let receiver = if let Some(ref receiver) = receiver {
        let receiver = deps.api.addr_validate(receiver)?;
        ensure!(
            info.sender == config.pool_manager_addr || info.sender == receiver,
            ContractError::Unauthorized
        );
        receiver
    } else {
        info.sender.clone()
    };

    // Check position limit early to fail fast
    validate_positions_limit(deps.as_ref(), &receiver, true)?;

    // Rest of the function...
}
```

🔗 [15] Fix incorrect documentation for position creation authorization check

Take a look at the documentation for the check provided during creation of positions:

</contracts/farm-manager/src/position/commands.rs#L41-L56>

```
// if a receiver was specified, check that it was the pool manager who
// is sending the message, as it has the possibility to lock LP tokens on
// behalf of the user
let receiver = if let Some(ref receiver) = receiver {
    let receiver = deps.api.addr_validate(receiver)?;
    ensure!(
        info.sender == config.pool_manager_addr || info.sender == receiver,
        ContractError::Unauthorized
    );

    receiver
} else {
    info.sender.clone()
};
```

The code comment, however, is misleading; considering whereas the documentation hint that only the pool manager can create positions, it actually checks that either the pool manager is the sender or the receiver of the position is the sender.

🔗 Impact

QA - wrong documentation.

🔗 Recommended Mitigation Steps

Update the comment to accurately reflect the authorization logic:

```
// if a receiver was specified, ensure the sender is either:
// 1. the pool manager (who can create positions on behalf of any user) or
// 2. the receiver themselves (users can create their own positions)
let receiver = if let Some(ref receiver) = receiver {
    let receiver = deps.api.addr_validate(receiver)?;
    ensure!(
        info.sender == config.pool_manager_addr || info.sender == receiver,
        ContractError::Unauthorized
    );

    receiver
} else {
    info.sender.clone()
};
```

🔗 [16] Emergency unlock penalty should not be allowed to be up to 100%

</contracts/farm-manager/src/helpers.rs#L177-L187>

```
/// Validates the emergency unlock penalty is within the allowed range (0-100%). Returns value it's valid
pub(crate) fn validate_emergency_unlock_penalty(
    emergency_unlock_penalty: Decimal,
) -> Result<Decimal, ContractError> {
    ensure!(
        emergency_unlock_penalty <= Decimal::percent(100),
        ContractError::InvalidEmergencyUnlockPenalty
    );

    Ok(emergency_unlock_penalty)
}
```

This validates the value that's to be passed as the emergency unlock penalty; however, it only checks if the value is above 100% before it errors out. If the value is set at 100% we can as well just consider the feature unsupported considering users would rather not withdraw than lose out everything on penalty.

🔗 Impact

QA, considering it's admin backed.

🔗 Recommended Mitigation Steps

Have a documented maximum value this can be set as and then instead have this check against this value.

🔗 [17] Inaccurate year calculation has been set in as AMM constants

NB: Similar issue also exists in the Position's manager scope.

</contracts/farm-manager/src/position/helpers.rs#L15-L17>

```
const SECONDS_IN_DAY: u64 = 86400;  
const SECONDS_IN_YEAR: u64 = 31556926;
```

The code snippet located [here](#) defines constants for a day and a month in seconds:

```
pub const LP_SYMBOL: &str = "LP";  
pub const DAY_IN_SECONDS: u64 = 86_400u64;  
pub const MONTH_IN_SECONDS: u64 = 2_629_746u64;
```

The protocol intends for 12 months to equal one year. However, the defined `MONTH_IN_SECONDS` value leads to an inaccurate year calculation.

Current Calculation:

Using the current `MONTH_IN_SECONDS`:

```
Seconds in a "year" (using current value) = 12 * 2_629_746 seconds  
                                           = 31_556_952 seconds
```

Expected Calculation:

A more accurate calculation for the number of seconds in a year considers the average length of a year, including leap years, which is approximately 365.25 days.

```
Seconds in a year = 365.25 days * 86_400 seconds/day  
                  = 31_557_600 seconds
```

Difference:

The difference between the current calculation and the expected value is:

```
Difference = 31_557_600 seconds - 31_556_952 seconds  
           = 648 seconds
```

Over the course of a year, this difference amounts to 648 seconds or approximately 10.8 minutes.

Percentage Difference:

To better understand the magnitude of the error, we can calculate the percentage difference:

```
Percentage Difference = (Difference / Expected Seconds in a Year) * 100  
                     = (648 / 31_557_600) * 100  
                     ≈ 0.00205%
```

While the percentage difference is small, it can still accumulate over longer periods and impact time-sensitive calculations.

Impact

This discrepancy, while seemingly minor, can lead to measurable inaccuracies in time-sensitive operations within the AMM. This is particularly relevant for:

- **Reward Distribution:** If rewards are distributed based on time-locked assets, users might receive slightly less or more than expected over longer periods.
- **Fee Calculations:** Fees based on duration might be slightly miscalculated, affecting revenue.
- **Time-Weighted Averages:** Price oracles that rely on time-weighted averages could be skewed, impacting trading decisions.

Recommended Mitigation Steps

Update the `MONTH_IN_SECONDS` constant to a more accurate representation. The simplest approach is to divide the accurate seconds in a year by 12:

```
pub const MONTH_IN_SECONDS: u64 = 31_557_600u64 / 12; // 2_629_800
```

🔗 [18] Inconsistent farm management permissions in sister operations

Take a look at the following code in `commands.rs`:

```
// In close_farm function - contract owner can close farms
ensure!(
    farm.owner == info.sender || cw_ownable::is_owner(deps.storage, &info.sender)?,
    ContractError::Unauthorized
);

// In expand_farm function - only farm owner can expand
ensure!(farm.owner == info.sender, ContractError::Unauthorized);
```

The contract implements inconsistent permission checks for farm management operations. While the contract owner has the ability to close any farm through the `close_farm` function, they are not granted the same privilege to expand farms through the `expand_farm` function.

This creates an asymmetric permission model where:

1. Farm owners can both close and expand their farms.
2. Contract owners can close any farm but cannot expand them.

This inconsistency could lead to issues in emergency situations where the contract owner needs to manage farms holistically but is limited by the permission model.

🔗 Recommended Mitigation Steps

Make the permission model consistent by adding contract owner check to `expand_farm`:

```
// Add contract owner check to expand_farm
ensure!(
    farm.owner == info.sender || cw_ownable::is_owner(deps.storage, &info.sender)?,
    ContractError::Unauthorized
);
```

🔗 [19] Consider conjugating the pool asset lengths validation when creating a pool

The pool creation function performs multiple separate validation checks for asset parameters that could be combined into a single check for better gas efficiency:

</contracts/pool-manager/src/manager/commands.rs#L88-L103>

```
// Ensure that the number of assets and decimals match, and that they are not empty
ensure!(
    !asset_denoms.is_empty()
    && asset_denoms.len() >= MIN_ASSETS_PER_POOL
    && asset_denoms.len() == asset_decimals.len(),
    ContractError::AssetMismatch
);

// Ensure that the number of assets is within the allowed range
ensure!(
    asset_denoms.len() <= MAX_ASSETS_PER_POOL,
    ContractError::TooManyAssets {
        assets_provided: asset_denoms.len(),
    }
);
```

The code performs two separate `ensure!` checks:

- First checks for empty array, minimum assets, and matching lengths.
- Second checks for maximum assets.

This requires:

- Two separate `ensure!` macro expansions.
- Multiple length checks on `asset_denoms`.
- Two separate error paths.

🔗 Impact

Low severity. While functionally correct, this approach:

Uses more computability than necessary due to redundant length checks due to multiple `ensure!` expansions and makes code slightly less maintainable with split validation logic.

🔗 Recommended Mitigation Steps

Combine the validations into a single `ensure!` check:

Pseudo Code:

```
ensure!(
  !asset_denoms.is_empty()
  && (MIN_ASSETS_PER_POOL..MAX_ASSETS_PER_POOL).contains(&asset_denoms.len())
  && asset_denoms.len() == asset_decimals.len(),
  ContractError::InvalidAssetConfiguration {
    assets_provided: asset_denoms.len(),
    min_assets: MIN_ASSETS_PER_POOL,
    max_assets: MAX_ASSETS_PER_POOL
  }
)
```

🔗 [20] Fix typos

[/contracts/farm-manager/src/position/commands.rs#L24-L109](#)

► Details

Function is used when creating new positions; however, in the call to validate the amount of open positions we instead document that we are checking the amount of closed positions.

🔗 Recommended Mitigation Steps

Apply [these changes](#):

► Details

🔗 [21] Remove redundant checks when withdrawing a position

Take a look at the position withdrawal logic [here](#):

```
// First check for expiring_at
ensure!(position.expiring_at.is_some(), ContractError::Unauthorized);

// Second check which includes the same is_some() check
ensure!(
  position.is_expired(current_time),
  ContractError::PositionNotExpired
);
```

The `is_expired` function in `farm_manager.rs` already checks for `expiring_at.is_some()`:

```
pub fn is_expired(&self, current_time: u64) -> bool {
```

```
    self.expiring_at.is_some() && self.expiring_at.unwrap() <= current_time
}
```

This creates:

1. Redundant checks for `expiring_at.is_some()`.
2. Inconsistent error messages for the same condition.
3. Unnecessary code complexity.

🔗 Recommended Mitigation Steps

Remove the first check and rely only on `is_expired`:

```
ensure!(
    position.is_expired(current_time),
    ContractError::PositionNotExpired
);
```

🔗 [22] Wrong code/documentation about fee

[/packages/amm/src/fee.rs#L25-L32](#)

```
/// Checks that the given [Fee] is valid, i.e. it's lower or equal to 100%
pub fn is_valid(&self) -> StdResult<()> {
    if self.share >= Decimal::percent(100) {
        return Err(StdError::generic_err("Invalid fee"));
    }
    Ok(())
}
```

The intended logic is to allow the fee be as high as 100% as hinted by the comment; however, we revert even when the fee is 100%, cause the check is inclusive.

🔗 Recommended Mitigation Steps

```
- if self.share >= Decimal::percent(100) {
+ if self.share > Decimal::percent(100) {
```

🔗 [23] Use `assert_admin` or remove it since we have the `cw-ownable` being used

[/contracts/pool-manager/src/helpers.rs#L462-L476](#)

```
/// This function compares the address of the message sender with the contract admin
/// address. This provides a convenient way to verify if the sender
/// is the admin in a single line.
pub fn assert_admin(deps: Deps, env: &Env, sender: &Addr) -> Result<(), ContractError> {
    let contract_info = deps
        .querier
        .query_wasm_contract_info(env.contract.address.clone())?;
    if let Some(admin) = contract_info.admin {
        if sender != deps.api.addr_validate(admin.as_str())? {
            return Err(ContractError::Unauthorized);
        }
    }
    Ok(())
}
```

This func is expected to compare the address of the message sender with the contract admin; however, scanning through the whole repo we can see that this function is never used.

We can confirm the fact that this method is never used by running this [search command](#).

🔗 Impact

QA, since the contract is using the `cw-ownable` crate for ownership management, which is a standard and well-tested solution in the CosmWasm ecosystem, it makes sense to stick with `cw_ownable::assert_owner` rather than using this custom `assert_admin` function.

🔗 Recommended Mitigation Steps

Use the method or remove it.

🔗 [24] Protocol should be deployment ready

Across scope there are multiple instances of core files that have not been implemented that are core to the product being live, this can be seen by searching `todo` through the repo, where we have the following results:

```
3 results - 3 files

docs/CODE_OF_CONDUCT.md:
  2
  3: todo

docs/CONTRIBUTING.md:
  2
  3: todo

docs/SECURITY.md:
  2
  3: todo
  4
```

To hint one, the security is a very core aspect of the docs and should be filled, as this is how other whitehat hackers can reach out to Mantra in the case they find a bug in live code.

The other two files should be implemented too before live prod.

🔗 Recommended Mitigation Steps

Implement these docs.

🔗 Disclosures

C4 is an open organization governed by participants in the community.

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.

Top